

분산처리

Multi-Core Architecture, Part II (latency/bandwidth issues)

+

Parallel Programming Abstractions



Computer & Ai

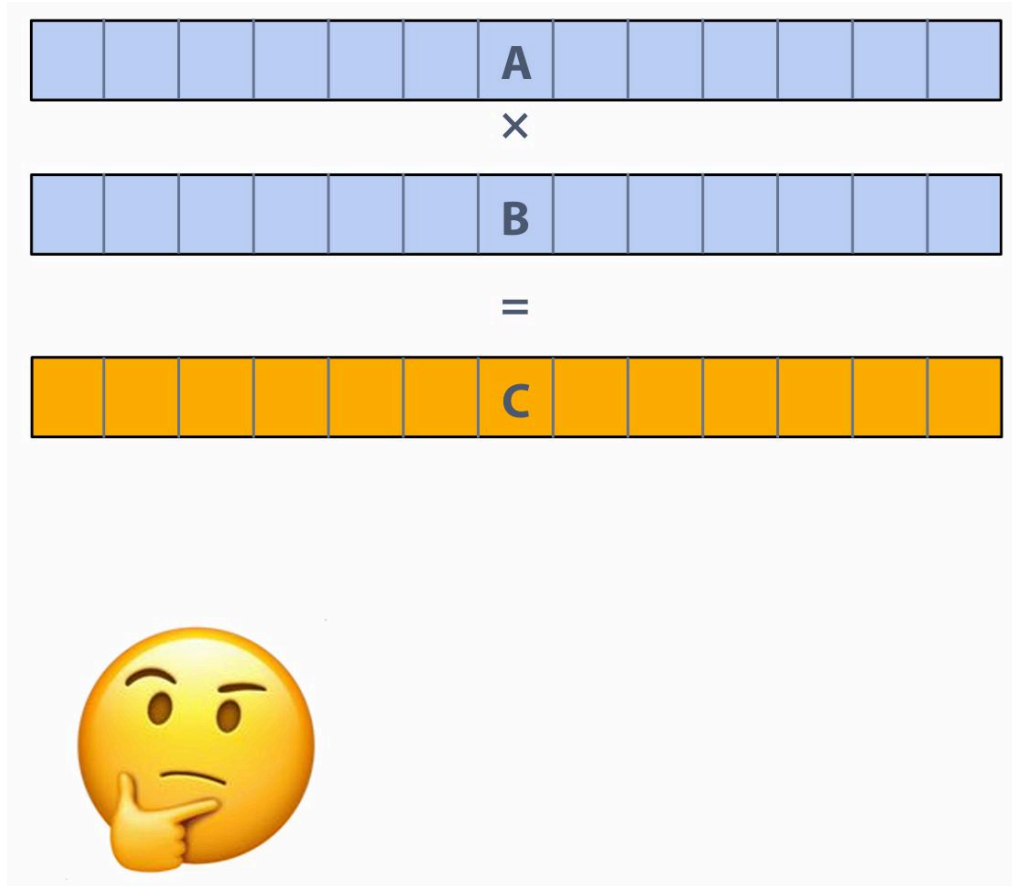
Department of Computer Engineering

사고 실험(생각하기)

두 벡터 A와 B의 요소별 곱셈 작업. 이 작업은 다음과 같은 단계로 이루어짐:

- 입력 A[i] 로드
- 입력 B[i] 로드
- $A[i] * B[i]$ 계산
- 결과를 C[i]에 저장

- 이 실험은 현대의 병렬 처리 지향 프로세서에서 실행하기에 적합한지 평가

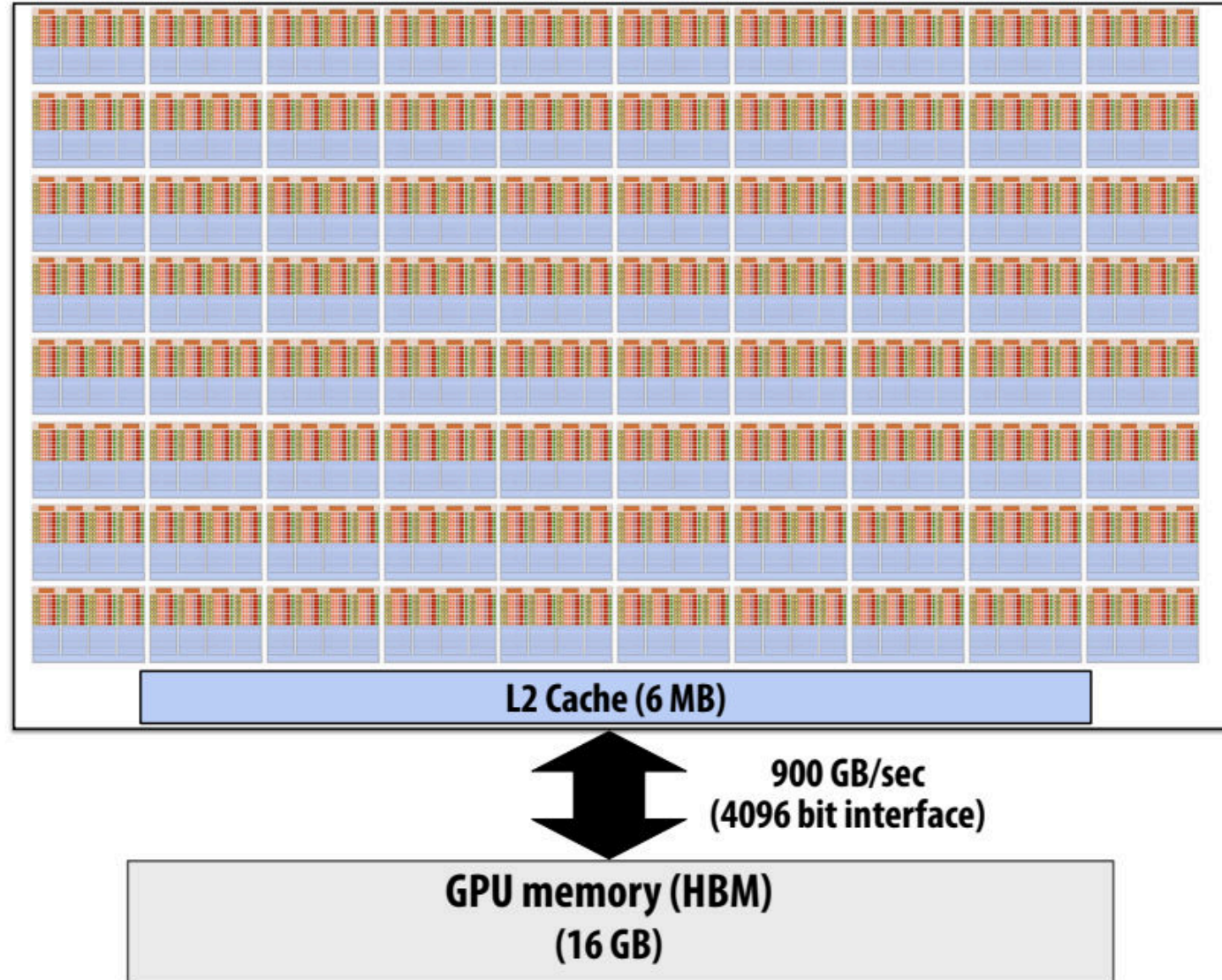


NVIDIA V100

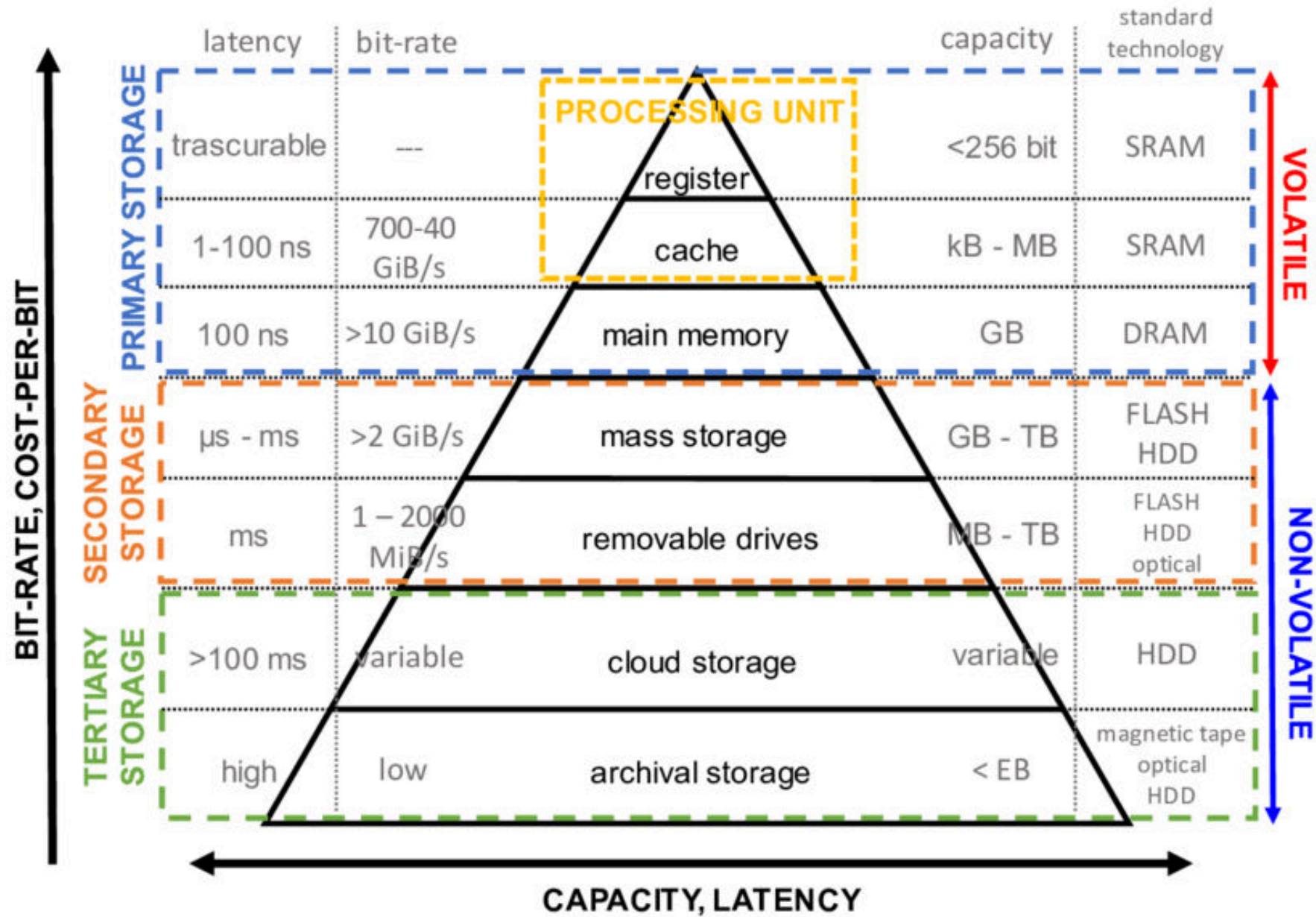
There are 80 SM cores on the V100:
 $80 \text{ SM} \times 64 \text{ fp32 ALUs per SM} = 5120 \text{ ALUs}$

모든 ALU에 매 클럭마다 데이터를 공급한다고 생각해 보세요.

- V100 GPU는 80개의 SM 코어를 가지고 있으며,
- 각 코어는 64개의 FP32 ALU를 포함하여 총 5120개의 ALU
- 이 모든 ALU에 매 클럭마다 데이터를 공급하는 것이 중요
- 이 작업은 메모리 대역폭에 의해 제한됨.
- 벡터의 각 요소를 곱하기 위해서는 세 개의 메모리 작업(12바이트)이 필요하며,
- V100 GPU는 클럭당 5120개의 FP32 곱셈을 수행 가능
- 이를 유지하려면 약 98TB/초의 대역폭이 필요.
- 하지만 실제로는 GPU 효율이 1% 미만이지만, 여전히 8코어 CPU보다 훨씬 빠름.
- 이 실험은 메모리 대역폭이 병목 현상이 되는 상황을 보여주며, 병렬 처리 시스템에서 메모리 대역폭을 극복하는 것이 중요한 과제임.
- 프로그램이 메모리에 자주 접근하지 않도록 최적화하는 것이 현대 프로세서의 효율적인 활용에 필수적임.

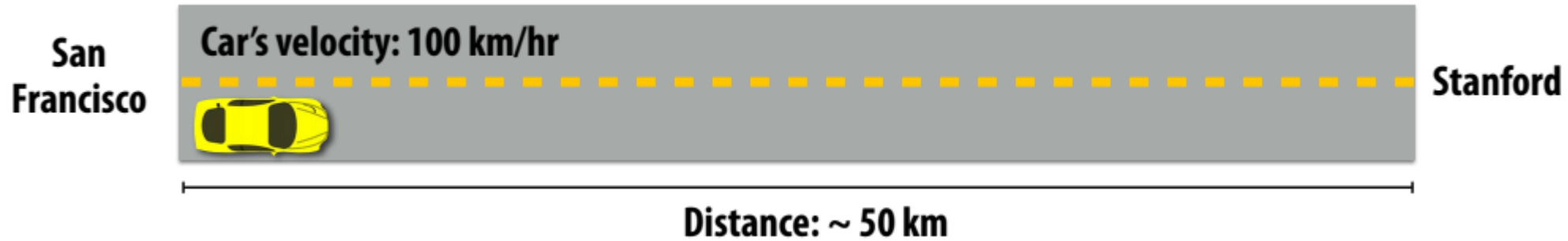


Understanding latency and bandwidth



Everyone wants to get to back to the South Bay!

- 고속도로의 한 차선에는 한 번에 한 대의 차량만 있다고 가정.
- 고속도로의 차량이 스탠포드에 도착하면 다음 차량이 샌프란시스코를 출발.

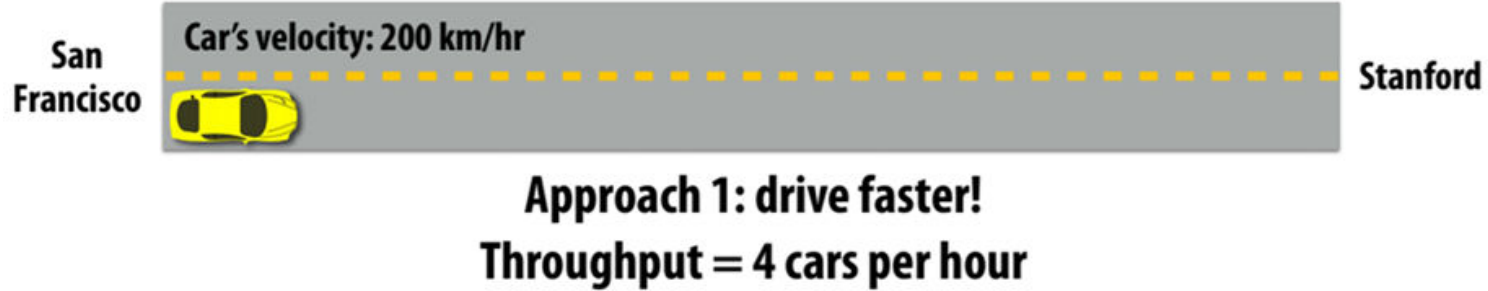


- 샌프란시스코에서 스탠포드까지 주행 대기 시간: 0.5시간
- 처리량: 시간당 차량 2대

Improving throughput(처리량 향상)

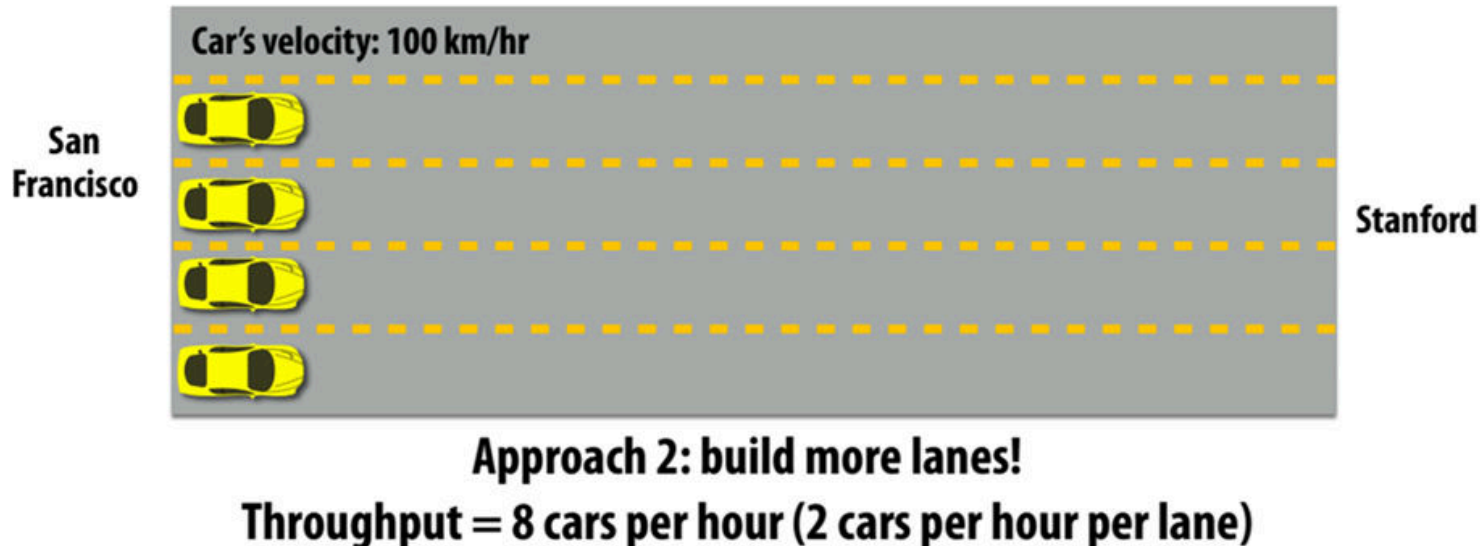
- 속도 증가:

- 차량의 속도를 높여 처리량을 증가. 예를 들어, 차량의 속도를 100 km/hr에서 200 km/hr로 증가시키면 처리량이 두 배로 증가.



- 차선 추가:

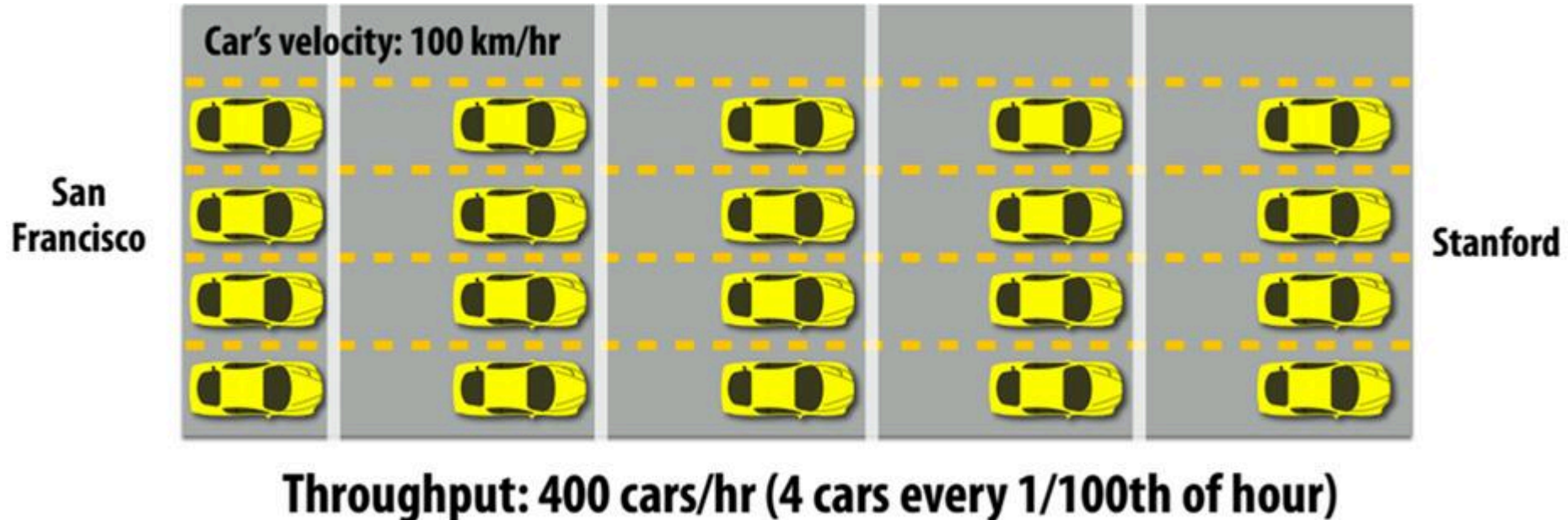
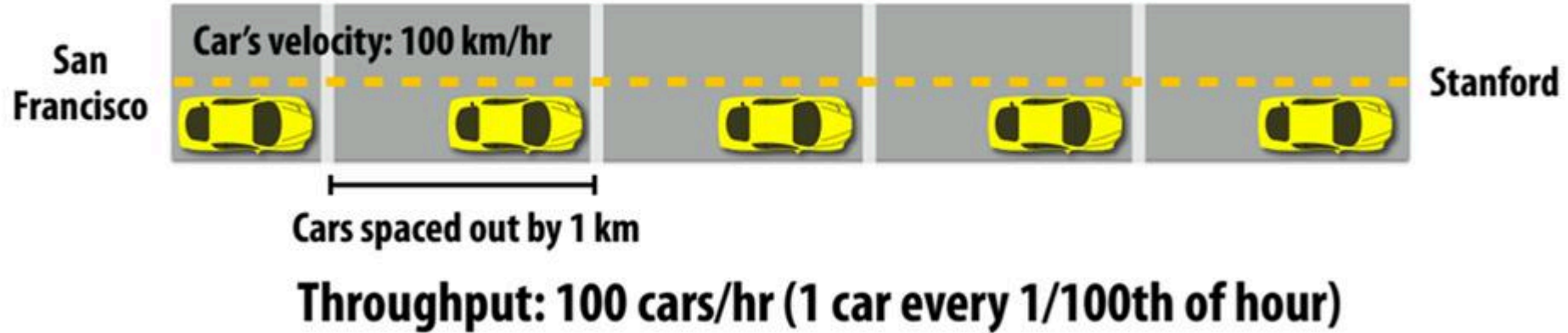
- 더 많은 차선을 추가하여 처리량을 증가. 예를 들어, 두 개의 차선을 추가하면 처리량이 두 배로 증가.



고속도로를 더 효율적으로 이용하기

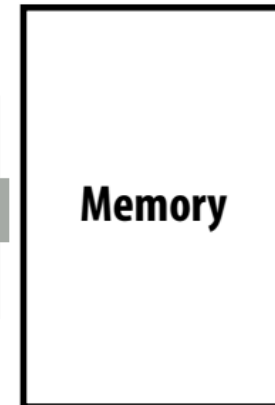
- 효율적인 사용:

- 고속도로를 더 효율적으로 사용하여 처리량을 증가시키는 방법. 예를 들어, 차량 간격을 줄여 더 많은 차량을 동시에 이동시킴



Terminology(용어)

- Memory bandwidth(메모리 대역폭)
 - The rate at which the memory system can provide data to a processor
(메모리 시스템이 프로세서에 데이터를 제공할 수 있는 속도)
 - Example: 20GB/s

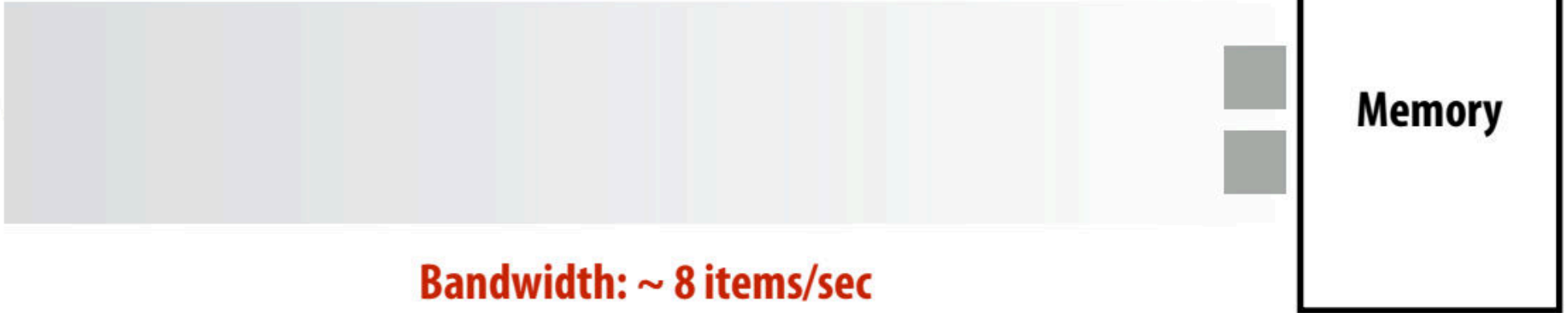


Bandwidth ~ 4 items/sec

Latency of transferring any one item: ~2 sec

Terminology(용어)

- Memory bandwidth(메모리 대역폭)
 - The rate at which the memory system can provide data to a processor
(메모리 시스템이 프로세서에 데이터를 제공할 수 있는 속도)
 - Example: 20 GB/s



Bandwidth: ~ 8 items/sec

Latency of transferring any one item: ~2 sec

■ 예: 빨래하기

작동: 빨래하기

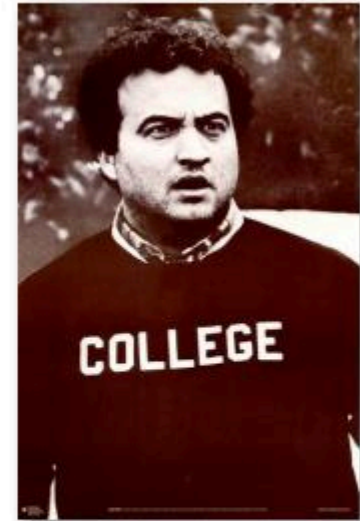
1. 빨래하기
2. 옷 말리기
3. 옷 개기



Washer
45 min



Dryer
60 min



College Student
15 min

Latency of completing 1 load of laundry = 2 hours

Increasing laundry throughput

목표: 많은 세탁물의 처리량 극대화

- 한 가지 방법은 실행 리소스를 중복하는 것:
- 세탁기 두 대, 건조기 두 대를 사용하고 친구에게 전화하기



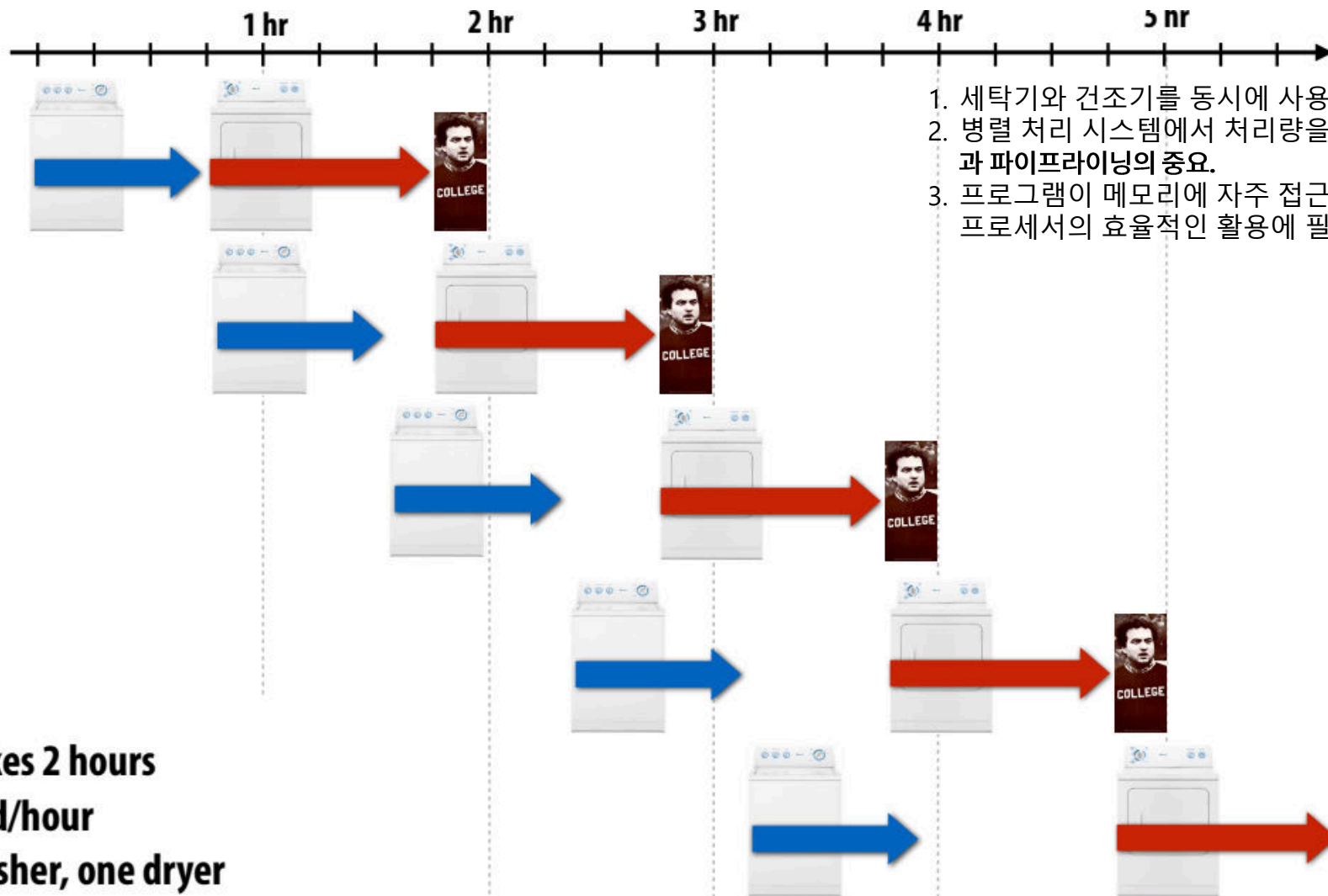
Latency of completing 2 loads of laundry = 2 hours

Throughput increases by 2x: 1 load/hour

Number of resources increased by 2x: two washers, two dryers

Pipelining laundry(파이프라인 세탁)

목표: 많은 세탁물의 처리량 극대화



1. 세탁기와 건조기를 동시에 사용하여 처리량을 두 배로 증가시킴
2. 병렬 처리 시스템에서 처리량을 증가시키기 위해서는 메모리 대역폭과 파이프라이닝의 중요.
3. 프로그램이 메모리에 자주 접근하지 않도록 최적화하는 것이 현대 프로세서의 효율적인 활용에 필수적

Latency: 1 load takes 2 hours

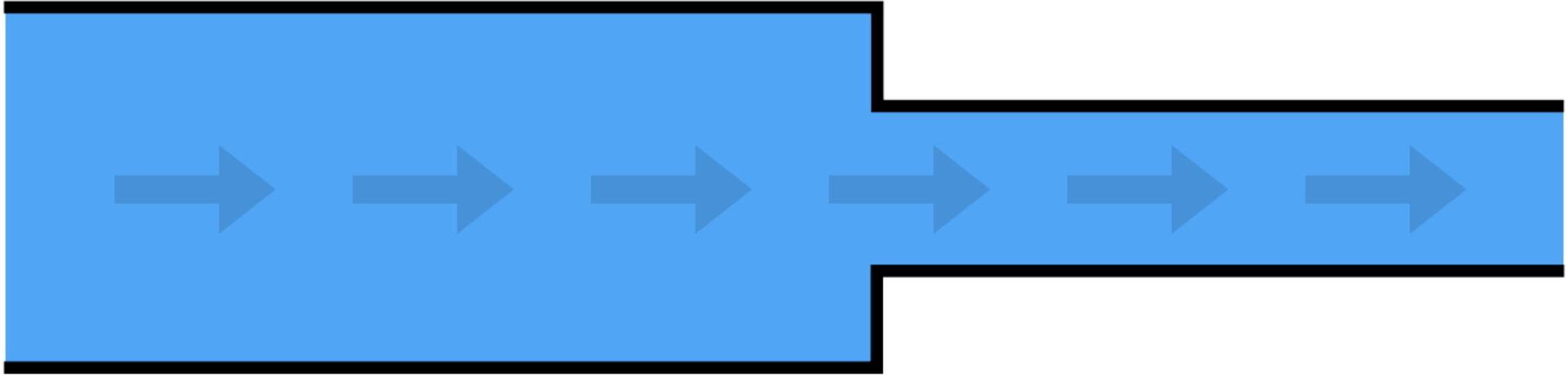
Throughput: 1 load/hour

Resources: one washer, one dryer

또 다른 예: 연결된 두 개의 파이프

Pipe 1: max flow 100 liters/sec

Pipe 2: max flow 50 liters/sec



파이프를 연결하면 시스템을 통해 밀어 넣을 수 있는 최대 유량은 얼마인가요?

50 liters/sec

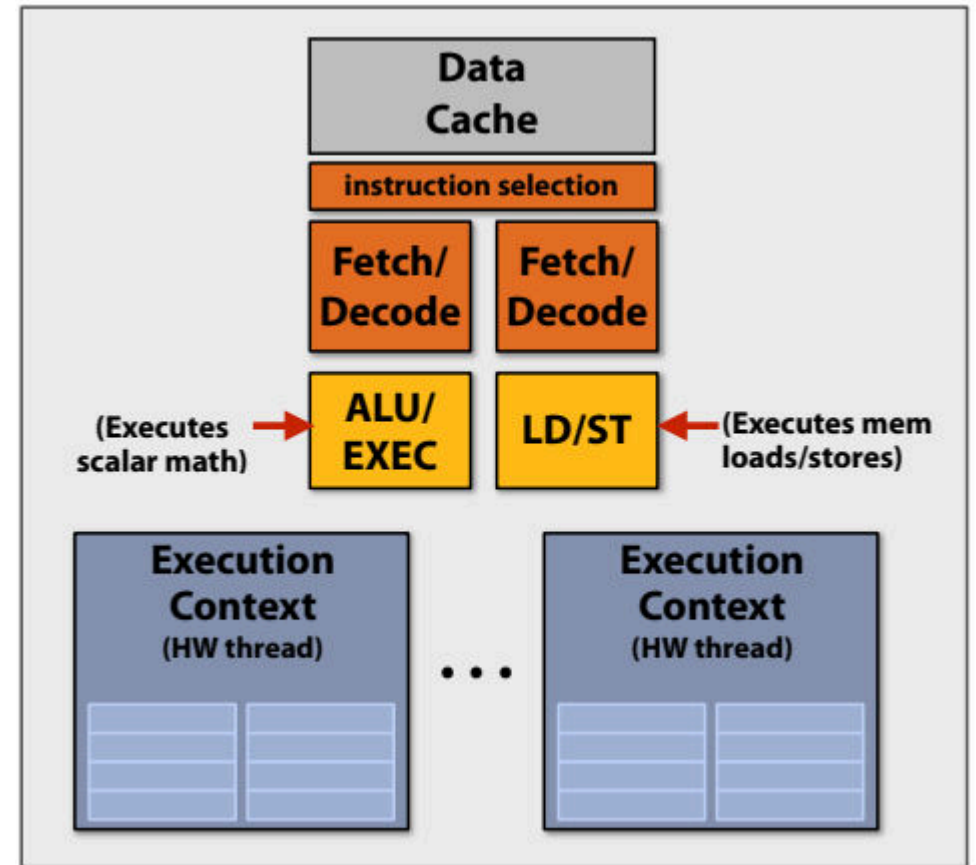
Applying this concept to a computer...

- 다음 세 가지 종속(의존) 명령어 순서를 반복하는 스레드를 실행하는 프로그램을 예로 들어 보자.

1. $X = \text{load } 64 \text{ bytes}$
2. $Y = \text{add } x + x$
3. $Z = \text{add } x + y$

- 멀티스레드* 코어의 여러 스레드에서 이 시퀀스를 실행한다고 가정해 보자:

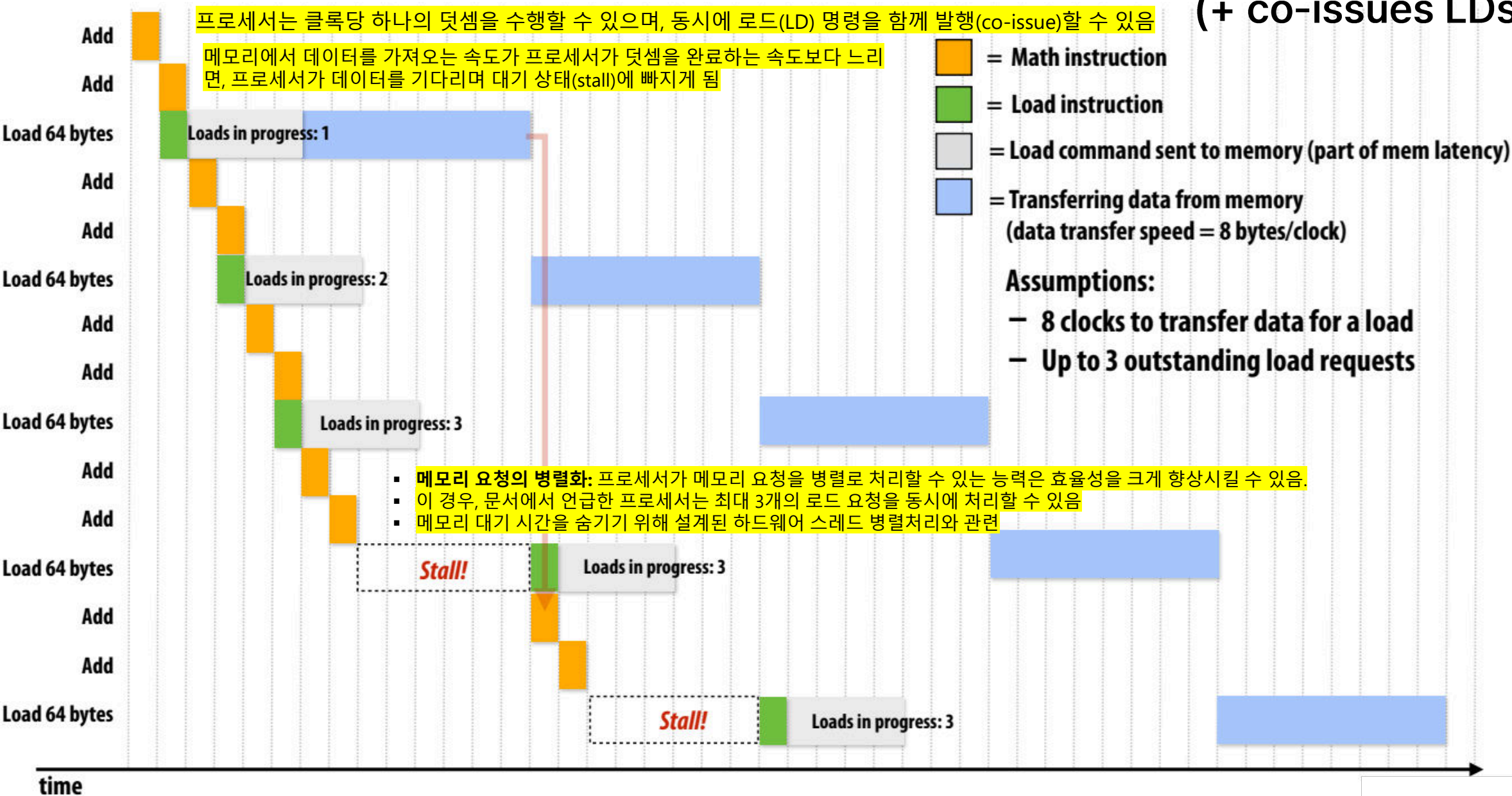
- 클럭당 하나의 수학 연산을 실행.
- 연산과 동시에 로드 명령을 내릴 수 있음.
- 메모리에서 클럭당 8바이트를 수신가능.



(* 메모리 지연 시간을 숨길 수 있는 하드웨어 스레드가 충분하다고 가정.)

Processor that can do one add per clock(한 클럭당 하나의 ADD 연산 가능프로세서)

(+ co-issues LDs)



Processor that can do one add per clock(클럭당 한 번 add()할 수 있는 프로세서)

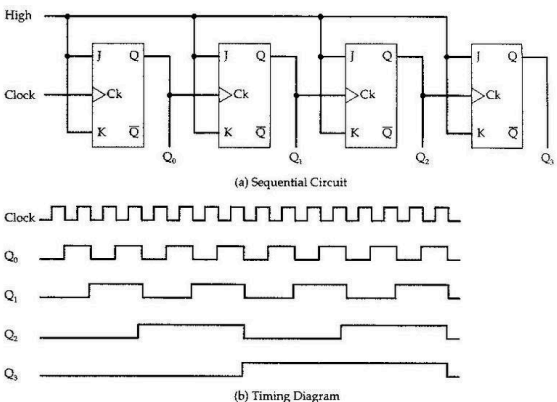
(+ co-issues LDs)

- LD(Load) 명령어와 Co-Issue 가능

- ADD 연산과 동시에 메모리에서 데이터를 로드하는 명령어(LD)도 실행 가능, 즉 ADD 연산과 LD 연산은 별개의 연산 유닛에서 수행되므로 서로 독립적.
- 동일한 클럭 사이클에서 ADD와 LD를 함께 실행(Co-Issue) 할 수 있음, 즉, 명령어가 독립적이면 한 사이클 내에서 병렬로 실행 가능.
- 실행 효율이 증가하고, 성능이 향상

- ILP(Instruction-Level Parallelism)를 활용

- 서로 의존성이 없는 명령어들을 동시에 실행하여 성능을 극대화



```
1. r1 = r2 + r3 // ADD 실행
2. r4 = MEM[addr] // LD 실행
3. r5 = r1 + r6 // ADD 실행 (이전 ADD의 결과를 사용)
```

이 경우, 명령어 실행 순서를 최적화하여 LD를 ADD와 동시에 실행가능

Clock Cycle	실행 명령어 1	실행 명령어 2 (Co-Issue)
Cycle 1	r1 = r2 + r3 (ADD)	r4 = MEM[addr] (LD)
Cycle 2	r5 = r1 + r6 (ADD)	-

- Co-Issue를 활용하면 실행 시간이 단축

- 일반적으로 ADD와 LD를 개별적으로 실행하면 3 클럭이 걸릴 수도 있지만, Co-Issue를 적용하면 2 클럭 만에 완료 가능.

- CPU 자원을 최대한 활용하여 병렬 처리

- 명령어 간 데이터 의존성을 최소화하고, 독립적인 명령어를 동시에 실행함으로써 효율 증가.

수학 명령어 완료 속도는 메모리 대역폭에 의해 제한

- 연산 성능(컴퓨팅 파워)이 메모리 대역폭에 의해 제한될 수 있음 → CPU가 수많은 연산을 처리할 능력이 있더라도, 메모리에서 데이터를 가져오는 속도가 충분하지 않으면 연산 속도가 제한



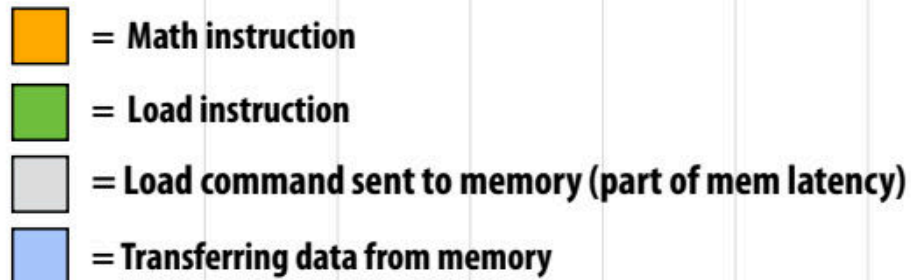
- 메모리 대역폭 병목현상(memory bandwidth bottleneck)으로 알려져 있으며, 과학 계산, 그래픽 연산, AI 모델 학습 등에서 자주 발생

가정:

- CPU는 클럭당 2개의 연산을 수행 가능.
- CPU 클럭 속도 = 2 GHz (즉, 초당 20억 클럭).
- 따라서, 이론적인 연산 속도 = 4 GFLOPS (초당 40억 번의 연산).
- 하지만, CPU가 데이터를 가져오는 속도가 제한적이면 실제 연산 속도는 이보다 훨씬 낮음.

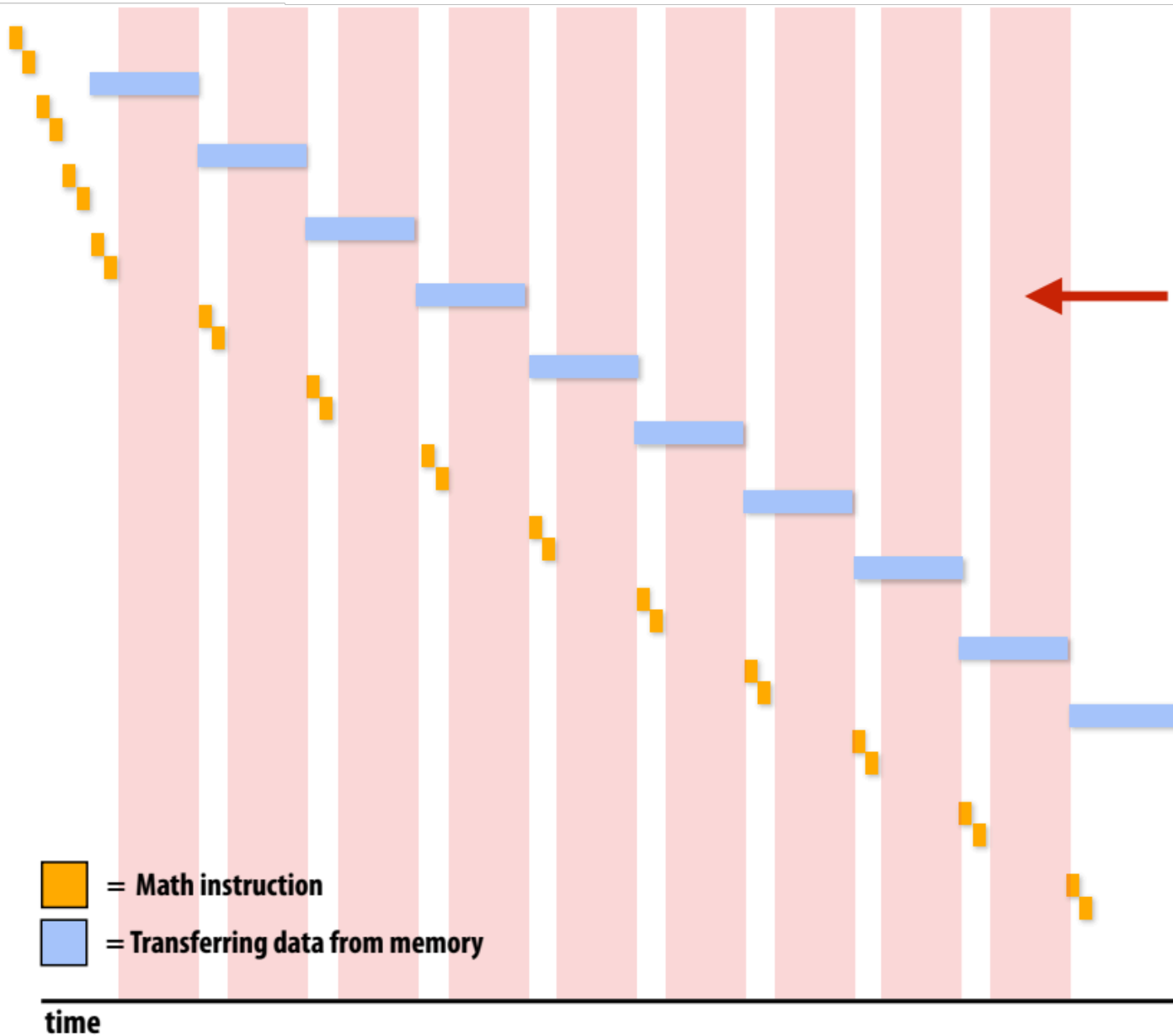
데이터 전송 속도가 클럭당 8바이트로 설정되어 있으며, 64바이트를 로드하려면 8 clock가 필요.

이 제한은 프로세서가 연속적인 수학 연산을 지속적으로 수행하는 데 필요한 데이터를 얼마나 빠르게 제공할 수 있는지 결정



time

수학 명령어 완료 속도는 메모리 대역폭에 의해 제한됨



메모리 대역폭 제한 실행!

명령어 실행 속도는 메모리가 데이터를 제공할 수 있는 속도에 따라 결정.

빨간색 영역:

코어가 다음 명령어를 위해 데이터를 기다리는 중

메모리는 100%의 시간 동안 데이터를 전송하고 있으며, 더 빠르게 데이터를 전송할 수 없음.

정상 상태에서의 코어 활용도 저하는 메모리 지연 시간이나 미결 메모리 요청의 수가 아니라 명령어와 메모리 처리량의 함수일 뿐이라는 점을 명심하세요.

수학 명령어 완료 속도는 메모리 대역폭에 의해 제한

메모리 대역폭 제한으로 성능 저하 (예)

가정:

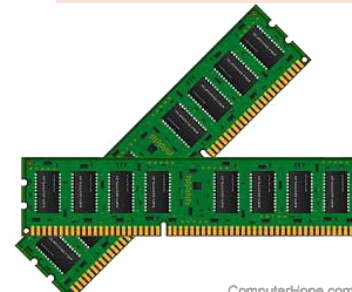
- CPU가 2개의 벡터를 더하는 코드 실행:

```
c
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

- 각 숫자는 64비트 (8바이트)
- A[i]와 B[i]를 로드하고, C[i]에 저장하려면 메모리 대역폭이 필요함
- 필요한 총 데이터 이동량:
 - A[i] 로드 (8B) + B[i] 로드 (8B) + C[i] 저장 (8B) = 총 24B per 연산
- 만약 메모리 대역폭이 24GB/s 라면?
 - CPU는 초당 1B 연산당 1GFLOPS 밖에 못 함 → CPU 성능보다 메모리 속도가 병목을 유발.

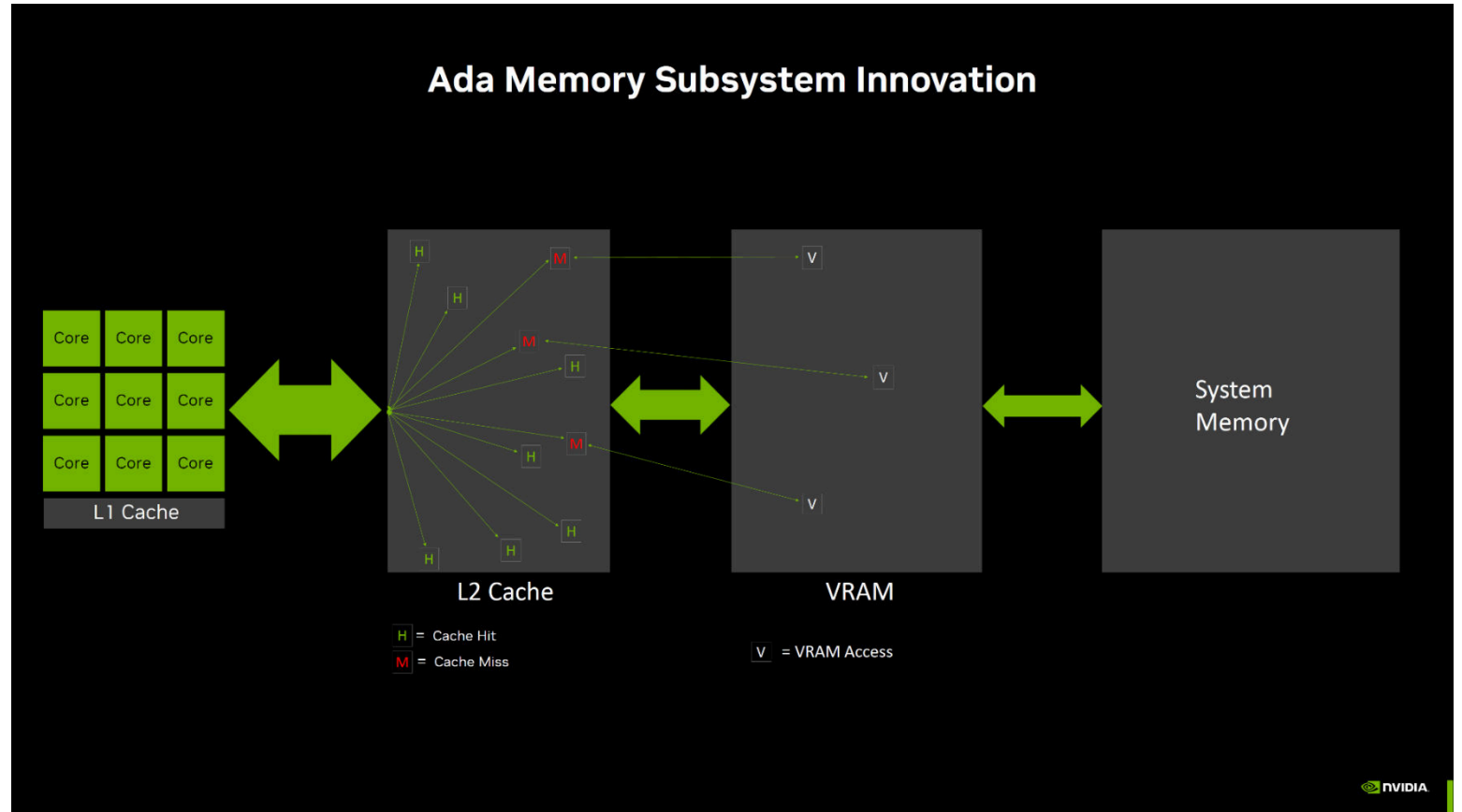
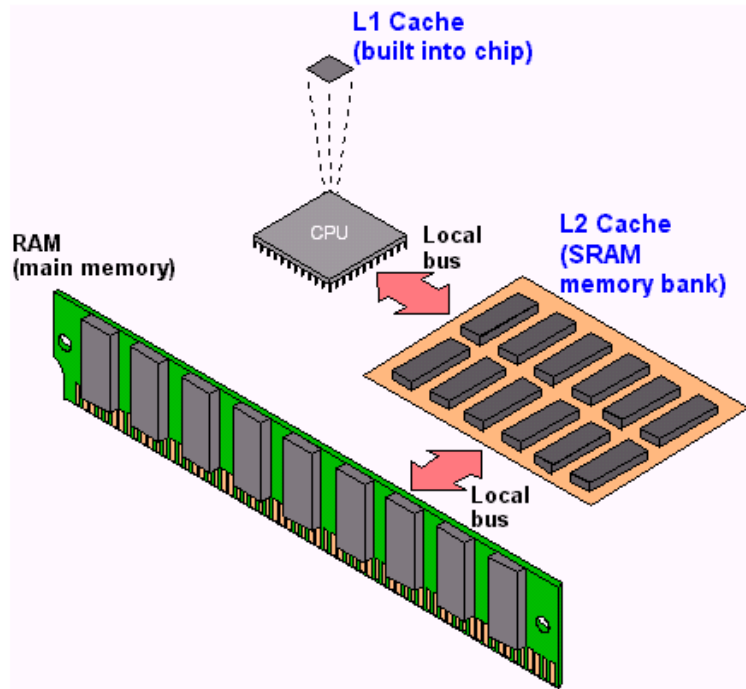
해결
방안

- 캐시 활용(Cache Optimization)
 - ✓ CPU의 L1/L2/L3 캐시를 최대한 활용하여 메모리 접근을 줄임.
 - ✓ 데이터 재사용률이 높은 알고리즘을 사용 (예: Blocking, Loop Tiling 기법).
- 메모리 인터리빙 (Memory Interleaving)
 - ✓ 여러 개의 메모리 뱅크를 동시에 활용하여 대역폭을 증가시킴.
- 벡터화(Vectorization) 및 SIMD 활용
 - ✓ ISPC, AVX 같은 벡터 연산을 활용하여 한 번에 많은 데이터를 처리함.
 - ✓ CPU의 SIMD 명령어를 사용하여 한 번의 메모리 접근당 더 많은 연산 수행.
- NUMA(Non-Uniform Memory Access) 최적화
 - ✓ 멀티코어 시스템에서는 메모리 접근 위치를 최적화하여 속도를 높임.



수학 명령어 완료 속도는 메모리 대역폭에 의해 제한됨.

From Computer Desktop Encyclopedia
© 1999 The Computer Language Co., Inc.



High bandwidth memories

최신 GPU는 프로세서 근처에 위치한 고대역폭 메모리를 활용

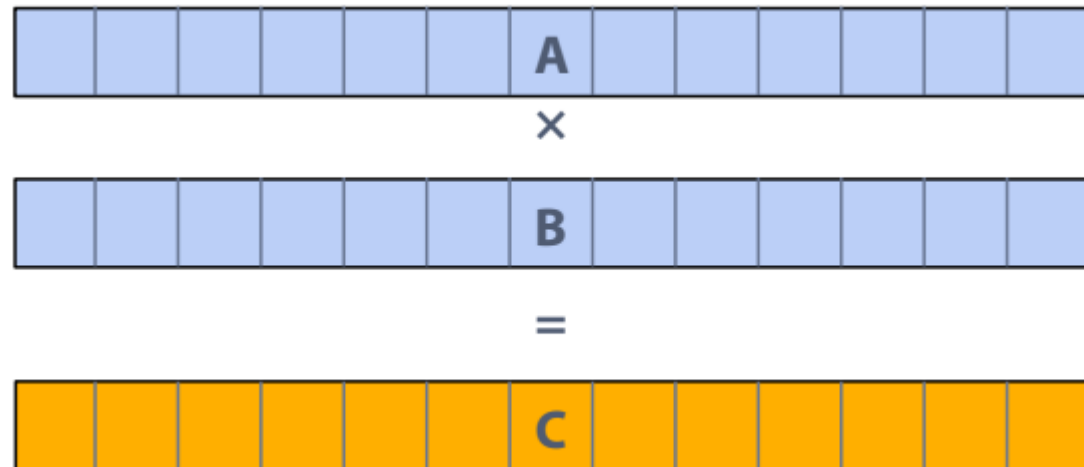
- **Example:**
 - V100 uses HBM2
 - 900 GB/s



Back to our thought experiment

- 작업: 두 벡터 A와 B의 요소별 곱하기
- 벡터에 수백만 개의 요소가 포함되어 있다고 가정.

- Load input A[i]
- Load input B[i]
- Compute $A[i] \times B[i]$
- Store result into C[i]



- 모든 MUL당 3개의 메모리 연산(12바이트)
- NVIDIA V100 GPU는 클럭당 5120개의 fp32 MUL을 수행가능 (@ 1.6GHz).
- 기능 유닛을 계속 사용하려면 초당 ~98TB의 대역폭 필요

<1% GPU 효율... 그래도 8코어 CPU보다 빨라야 합니다!

(76GB/sec 메모리 버스에 연결된 3.2GHz 제온 E5v4 8코어 CPU: 이 계산에서 약 3%의 효율성)

이 계산은 대역폭이 제한되어 있습니다!

- 프로세서가 너무 빠른 속도로 데이터를 요청하면 메모리 시스템이 이를 따라잡을 수 없음.
- 대역폭 한계를 극복하는 것은 최신 처리량 최적화 시스템(throughput-optimized systems)을 목표로 하는 소프트웨어 개발자가 직면한 가장 중요한 과제입니다.

In modern computing, bandwidth is the **critical** resource

◆ 성능이 우수한 병렬 프로그램은 다음을 수행.

- 메모리에서 데이터를 덜 자주 가져오도록 계산을 구성.
 - 동일한 스레드에서 이전에 로드한 데이터 재사용
(시간적 로컬리티(지역성) 최적화) → 데이터 재사용(temporal locality)
 - 스레드 간 데이터 공유(→ 스레드 간 협업)
- 값 저장/재로드 시 추가 연산 수행 선호(연산은 "무료")
- 요점: 최신 프로세서를 효율적으로 활용하려면 프로그램이 메모리에 자주 액세스하지 않아야 함.



현대 컴퓨팅에서 "대역폭(Bandwidth)"이 가장 중요한 자원. 즉, CPU나 GPU의 연산 속도보다 데이터를 얼마나 빠르게 가져올 수 있는지가 전체 성능을 결정짓는 핵심 요소

병목 현상 해결 전략

- 데이터 접근 횟수를 줄이는 프로그램 설계.
- 추가적인 산술 연산을 통해 값을 저장하거나 다시 로드할 필요를 없앴.
- 고대역폭 메모리 기술 사용(HBM2와 같은)

In modern computing, bandwidth is the **critical** resource

(1) 컴퓨팅 성능 vs. 메모리 대역폭의 차이

- CPU/GPU의 연산 성능은 기하급수적으로 증가하고 있음.
 - ❖ 예: 1990년대부터 현재까지 CPU의 연산 속도(FLOPS)는 1000배 이상 증가.
 - ❖ GPU는 더 빠르게 발전하여 병렬 연산 능력이 수천 배 향상됨.
- 하지만, 메모리 대역폭은 상대적으로 느리게 증가.
 - ❖ DRAM 및 메모리 인터페이스의 속도는 연산 속도만큼 빠르게 성장하지 못함.
 - ❖ 결과적으로, 연산을 처리할 데이터가 부족하여 CPU/GPU가 유휴 상태(Idle) 가 되는 경우가 많음.

(2) 메모리 대역폭이 병목(Bottleneck)이 되는 이유

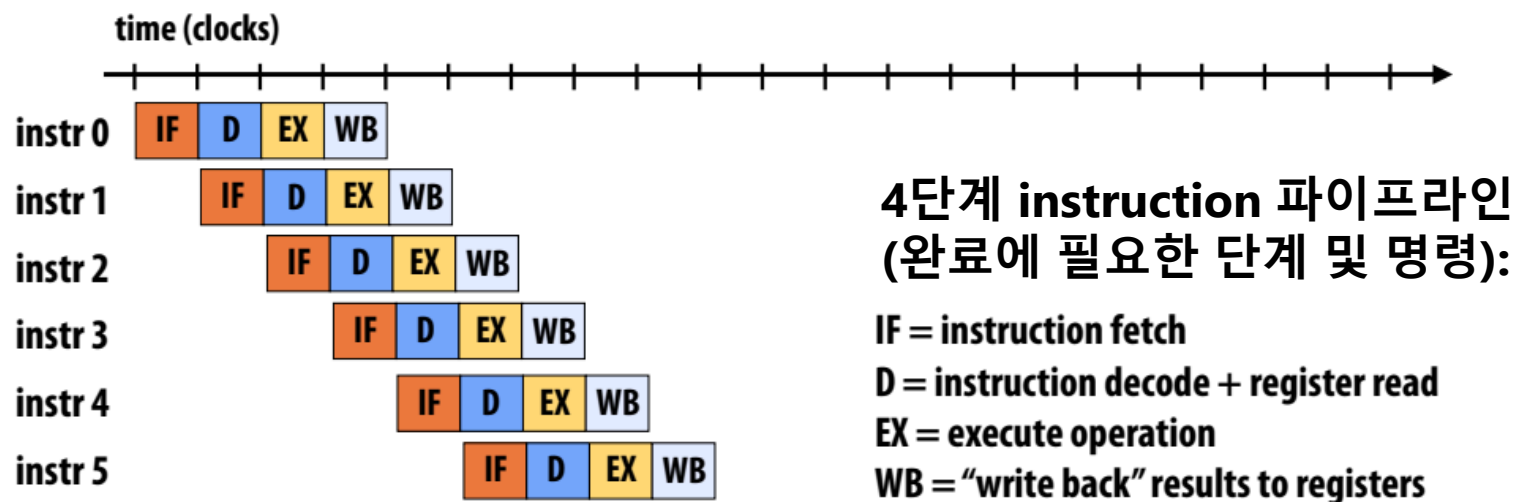
- 현대 CPU/GPU는 초당 수천억 번의 연산을 수행할 수 있음.
- 그러나, 데이터를 가져오는 속도(메모리 대역폭)가 이를 따라가지 못하면 실제 성능이 저하됨.
- 예를 들어, CPU가 초당 100개의 연산을 수행할 수 있지만, 메모리에서 초당 50개의 데이터만 제공되면,
 - ❖ 실제 성능 = 50개의 연산/초 (즉, 메모리 대역폭이 성능을 제한).

(3) 현대 컴퓨팅에서 대역폭이 중요한 이유

- CPU/GPU의 속도는 매우 빠름 → 하지만 데이터를 가져오는 것이 느리면 활용할 수 없음.
- AI, 빅데이터, 그래픽 처리, 시뮬레이션 등에서 대역폭이 중요한 자원이 됨.
- HPC (고성능 컴퓨팅), 클라우드 컴퓨팅, 머신러닝에서도 대역폭 최적화가 필수.

또 다른 파이프라이닝 예시: 명령어 파이프라인

- 많은 학생들이 프로세서가 어떻게 한 클럭에서 곱셈 연산을 완료할 수 있는지 종종 질문함.
- 코어가 클럭당 하나의 연산을 수행한다는 것은 **지연 시간이 아닌 명령어 처리량**을 의미.



지연 시간: 1개의 instruction에 4 cycles 소요

처리량: 사이클당 1개의 instruction

(예, 연속 명령어가 종속적인 경우 프로그램 정확성을 보장하기 위해 주의를 기울여야 합니다.)

실제 명령어 파이프라인은 최신 CPU에서 최대 20단계까지 (명령어에 따라) 가변 길이가 될 수 있음.

Part 2:

Abstraction vs. implementation

추상화의 Semantic(meaning)와 구현의 세부 사항을 혼동하는 것은
이 과정에서 흔히 발생하는 혼란의 원인이다.

추상화(Abstraction)와 구현(Implementation)의 차이점

핵심주제: 고수준 프로그래밍 모델과 실제 하드웨어에서의 동작 방식이 어떻게 다른지를 살펴보자

- 소프트웨어 개발자는 고수준의 추상화된 모델을 사용하지만, 실제 하드웨어에서는 다른 방식으로 동작할 수 있음.
- 성능을 최적화하려면 하드웨어의 실제 동작(Implementation)을 이해하는 것이 중요함.

추상화(Abstraction)란 무엇인가??

(1) 추상화 (Abstraction)

- 프로그래밍 모델이 하드웨어의 복잡성을 숨기는 과정.
- 예를 들어, C/C++에서 for 루프를 작성하면 내부적으로는 CPU에서 여러 개의 명령어로 변환됨.
- 프로그래머는 어떤 CPU 명령어가 실행되는지 신경 쓰지 않아도 됨.

c

```
for (int i = 0; i < N; i++) {  
    C[i] = A[i] + B[i];  
}
```

옆 코드에서는 단순한 반복문(for 루프)만 보이지만, 실제 CPU에서는 벡터화(Vectorization) 또는 파이프라이닝(Pipelining) 등의 최적화가 적용될 수도 있음.

추상화(Abstraction)와 구현(Implementation)의 차이점

구현 (Implementation) 이란 무엇인가??

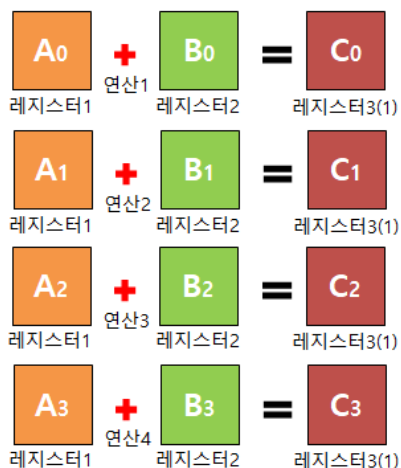
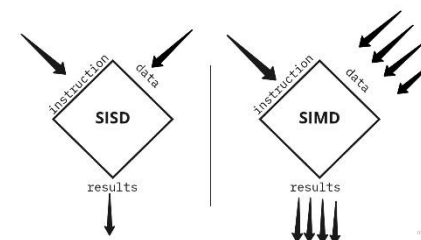
(2) 구현 (Implementation)

- 실제 하드웨어에서 코드가 어떻게 실행되는지를 의미.
- CPU/GPU가 제공하는 병렬 실행 방식, 캐시 구조, 명령어 파이프라인 등에 따라 성능이 달라질 수 있음.

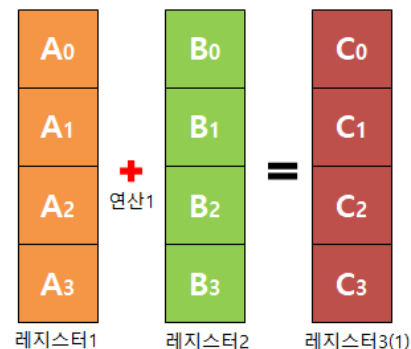
```
c
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

예제 (하드웨어 레벨에서의 실행 방식)

- 위의 for 루프가 단순한 스칼라 연산(Scalar Operation) 으로 실행될 수도 있고,
- SIMD(Vectorization)를 사용하여 한 번에 여러 개의 연산을 수행할 수도 있음.
- 하드웨어가 특정 최적화를 적용하지 않으면 생각보다 느리게 실행될 수도 있음.



SISD 연산



SIMD 연산

```
void add(int * inA, int * inB, int * res)
{
    res[0] = inA[0] + inB[0];
    res[1] = inA[1] + inB[1];
    res[2] = inA[2] + inB[2];
    res[3] = inA[3] + inB[3];
}

int _tmain(int argc, _TCHAR* argv[])
{
    int a[4] = { 1, 2, 3, 4 };
    int b[4] = { 5, 6, 7, 8 };
    int c[4];

    add(a, b, c);

    return 0;
}
```

SISD 연산

```
void add(int * inA, int * inB, int * res)
{
    __m128i a = _mm_load_si128((__m128i*)inA);
    __m128i b = _mm_load_si128((__m128i*)inB);
    __m128i c = _mm_add_epi32(a, b);
    _mm_store_si128((__m128i*)res, c);
}

int _tmain(int argc, _TCHAR* argv[])
{
    declspec(align(16)) int a[4] = { 1, 2, 3, 4 };
    declspec(align(16)) int b[4] = { 5, 6, 7, 8 };
    declspec(align(16)) int c[4];

    add(a, b, c);

    return 0;
}
```

SIMD 연산

추상화(Abstraction)와 구현(Implementation)의 차이점

예제 설명

예제 1: 벡터화(Vectorization) 적용 여부

(1) 추상화된 코드

```
for (int i = 0; i < N; i++) {  
    C[i] = A[i] + B[i];  
}
```

옆 코드만 보면 한 번에 한 개의 연산(Scalar Processing) 만 실행될 것처럼 보임.

(2) 실제 하드웨어에서의 실행 (SIMD 적용)

➢ CPU가 벡터 명령어(AVX, SSE)를 지원하는 경우, for 루프를 벡터화하여 한 번에 여러 개의 연산을 병렬 실행할 수 있음.

assembly

```
vmovaps xmm0, [A]    ; A 배열에서 데이터 로드  
vaddps  xmm0, xmm0, [B] ; A + B 벡터 연산 수행  
vmovaps [C], xmm0    ; 결과를 c 배열에 저장
```

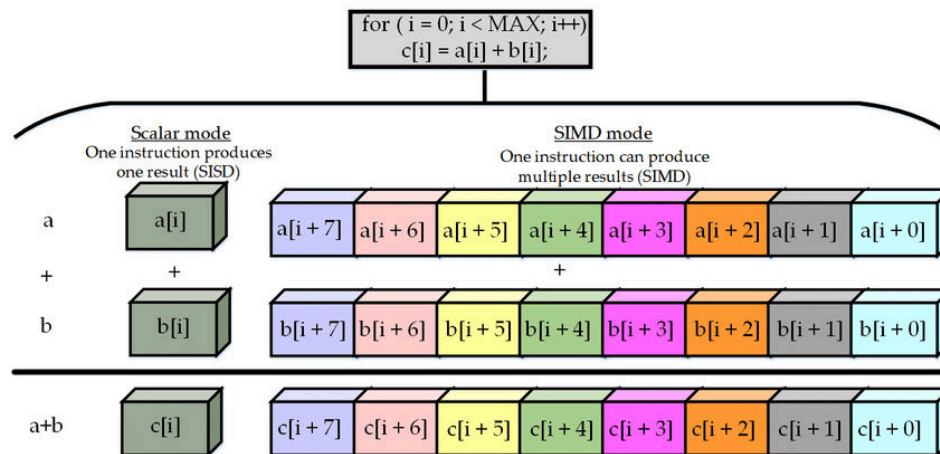
위처럼 SIMD(Vector Processing) 를 사용하면 한 번에 4~8개씩 연산 가능, 즉 성능이 4~8배 향상 될 수 있음.

추상화(Abstraction)와 구현(Implementation)의 차이점

(3) 구현 방식에 따른 성능 차이

실행 방식	연산 속도	특징
스칼라(Scalar)	느림	한 번에 1개 처리
벡터화(SIMD)	빠름	한 번에 여러 개 처리
멀티스레드	더 빠름	여러 개의 코어 사용

즉, 단순한 코드(추상화)라도 실제 하드웨어에서 어떻게 실행되느냐(구현)에 따라 성능이 크게 달라질 수 있음.



추상화(Abstraction)와 구현(Implementation)의 차이점

예제 2: 메모리 접근 방식 차이 (캐시 활용)

(1) 추상화된 코드

```
c
for (int i = 0; i < N; i++) {
    sum += A[i];
}
```

프로그래머는 $A[i]$ 값을 읽어서 sum 에 더하는 단순한 연산을 한다고 생각.

(2) 실제 하드웨어 구현

- 만약 배열 A 가 메모리 상에서 연속적이지 않거나 캐시 최적화가 되어 있지 않다면, CPU가 메모리에서 데이터를 가져오는 데 시간이 오래 걸림.
- 캐시 미스(Cache Miss) 발생 시 성능 저하.
- 이를 방지하려면, 캐시 친화적인 접근(Cache-Friendly Access Pattern) 을 고려해야 함.

💡 즉, 코드는 단순해 보여도, 실제 CPU가 데이터를 어떻게 가져오느냐(캐시 활용 여부)에 따라 성능 차이가 발생.

(1) 벡터화(Vectorization) 활용

- SIMD(AVX, SSE) 명령어를 활용하여 한 번에 여러 연산을 수행하면 성능 향상.
- 예제: ISPC (Intel SPMD Program Compiler) 를 사용하여 자동 벡터화.

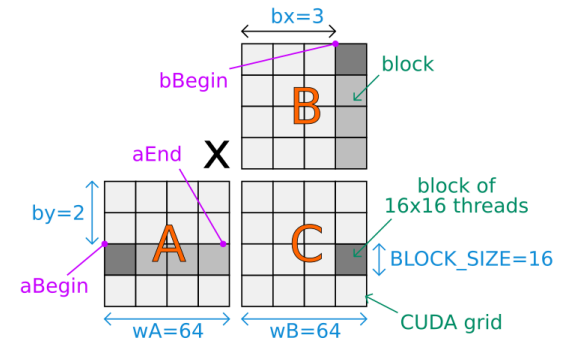
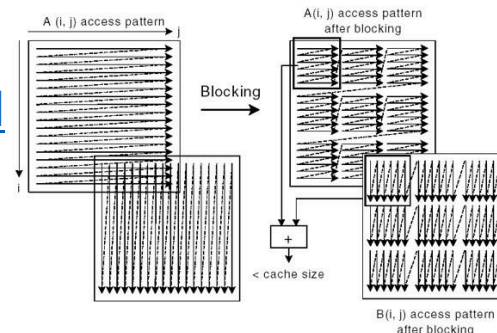
```
C  
  
foreach(i = 0 ... N) {  
    C[i] = A[i] + B[i];  
}
```

옆 코드에서 foreach를 사용하면 자동으로 벡터화됨.

(2) 캐시 최적화 (Cache Optimization)

- 데이터를 연속적인 메모리 레이아웃으로 정렬하여 캐시 미스(Cache Miss) 최소화.
- Loop Blocking (타일링 기법) 을 적용하여 캐시 히트(Cache Hit)율을 높임.

Loop Optimizations Where Blocks are Required



(3) 병렬 처리 (Parallelization) 활용

- CPU의 멀티코어, 멀티스레드 환경에서 **OpenMP, CUDA** 등의 기술을 사용하여 병렬 실행.

```
c
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

OpenMP를 사용하면 여러 개의 CPU 코어를 활용하여 병렬 연산 가능.

- 추상화(Abstraction)된 코드는 단순하지만, 실제 하드웨어에서 다르게 실행될 수 있음.
- 벡터화, 캐시 최적화, 병렬 처리를 적용하면 성능을 크게 향상시킬 수 있음.
- 최적의 성능을 얻기 위해서는 "**구현(Implementation)**"을 깊이 이해하는 것이 필수적.

- **ISPC (Intel SPMD Program Compiler)** 는 Intel이 개발한 병렬 프로그래밍을 위한 컴파일러.
 - SPMD (Single Program, Multiple Data) 모델을 기반으로 하며, **CPU의 SIMD(Vector) 명령어를 자동으로 활용할 수 있도록 설계됨**
 - ISPC를 사용하면 CPU에서 **벡터화(SIMD)** 및 병렬 실행을 쉽게 활용할 수 있음. 즉, C/C++ 코드보다 더 높은 성능을 자동으로 달성할 수 있음.
- ✓ **(1) SPMD (Single Program, Multiple Data) 모델 기반**
 - SPMD는 "하나의 프로그램이 여러 개의 데이터에 대해 동시에 실행되는 방식".
 - GPU에서 사용하는 **CUDA, OpenCL**과 유사하지만 **CPU 환경에서 최적화됨**.
 - **foreach** 구문을 사용하여 **자동으로 병렬 실행**할 수 있음.
 - ✓ **(2) SIMD(Vectorization) 자동 지원**
 - ISPC는 **CPU의 SIMD 명령어(SSE, AVX, AVX-512 등)**를 자동으로 활용.
 - 벡터화(Vectorization) 최적화를 직접 하지 않아도 **고성능을 쉽게 얻을 수 있음**.
 - ✓ **(3) C/C++과 쉽게 통합 가능**
 - ISPC 프로그램은 **C/C++ 코드와 함께 사용** 가능하며, C++ 함수에서 ISPC 함수를 호출할 수 있음.

ISPC 코드 예제

일반적인 C 코드 (벡터화 없음)

```
c
void add_arrays(float* a, float* b, float* c, int N) {
    for (int i = 0; i < N; i++) {
        c[i] = a[i] + b[i];
    }
}
```

위 코드는 스칼라 연산으로 실행되며,
SIMD 최적화가 적용되지 않음.

ISPC를 활용한 코드 (자동 벡터화)

```
c
export void add_arrays(uniform float* a, uniform float* b, uniform float* c, uniform int N) {
    foreach (i = 0 ... N) {
        c[i] = a[i] + b[i];
    }
}
```

- foreach (i = 0 ... N) 구문을 사용하면, 자동으로 **SIMD 벡터 연산으로 변환**됨.
- Intel AVX, SSE 명령어를 활용하여 한 번에 여러 개의 데이터를 처리할 수 있음.

성능 차이 (C vs. ISPC)

실행 방식	연산 속도	특징
C (스칼라 연산)	느림	한 번에 1개 처리
C + OpenMP (멀티스레드)	빠름	여러 개의 CPU 코어 활용
ISPC (벡터화 + 병렬 처리)	매우 빠름	한 번에 여러 개 처리 (SIMD + 병렬 실행)

ISPC와 OpenMP 비교

특징	ISPC	OpenMP
사용 모델	SPMD (Single Program, Multiple Data)	Shared Memory 멀티스레딩
SIMD 자동화	✓ (자동 벡터화)	✗ (명시적으로 벡터화 필요)
멀티코어 활용	✓	✓
코드 복잡도	낮음 (간단한 foreach 사용)	높음 (pragma 사용 필요)
GPU 실행 가능 여부	✗ (CPU 전용)	✗ (CPU 전용)

• ISPC는 벡터화(Vectorization)에 강점이 있고, OpenMP는 멀티스레딩(Threading) 중심.

- 소프트웨어 개발자는 고수준의 추상화된 모델을 사용하지만, 실제 하드웨어에서는 다른 방식으로 동작할 수 있음.
- 성능을 최적화하려면 하드웨어의 실제 동작(Implementation)을 이해하는 것이 중요함.

Abstraction vs. implementation

의미론(**Semantics**): 프로그래밍 모델에서 제공하는 연산은 무엇을 의미하나요?

구현(**Implementation (aka scheduling)**): 병렬 머신에서 답을 어떻게 계산할 것인가요?

프로그램이 주어지고 사용된 연산의 semantic (meaning)가 주어졌을 때, 프로그램이 계산할 답은 무엇일까요?

- 프로그램의 연산은 어떤 (잠재적으로 병렬적인) 순서로 실행될까요?
- 각 스레드에서 어떤 연산을 계산할 것인가?
- 각 실행 단위? 벡터 명령어의 각 레인?

- 추상화는 병렬 프로그래밍 모델이 개발자에게 제공하는 개념적 틀.
- 이는 개발자가 "무엇을 해야 하는지(semantics)"를 정의하고, 어떻게 병렬 처리기를 활용할지를 고민할 필요 없이 높은 수준에서 코드를 작성할 수 있게 함.
 - 예: 프로그래머는 특정 데이터 집합에 대해 반복적으로 작업을 수행해야 한다는 것을 명시하지만, 작업의 배치(scheduling)나 스레드 분배 방식은 추상화 뒤에 숨겨져 있음.
- 구현은 실제 병렬 머신에서 추상화된 코드를 실행하기 위한 상세한 방법
 - 병렬 명령어의 순서, 각 명령어가 어떤 스레드에서 실행될지, 또는 SIMD 레인에서 어떻게 분할할지를 결정.
 - 스레드 및 하드웨어 리소스 분배: 구현은 각 계산 작업이 어떤 실행 유닛(execution unit), 스레드, 혹은 벡터 명령어 레인에서 실행될지를 관리.

학생으로서의 목표: 병렬 프로그래밍 모델이 어떻게 구현되는지에 대한 프로그램과 지식이 주어졌을 때, 프로그램의 각 단계에서 병렬 컴퓨터의 각 부분이 수행하는 작업을 머릿속으로 '추적'할 수 있어야 함.

(* 메모리 지연 시간을 숨길 수 있는 하드웨어 스레드가 충분하다고 가정.)

병렬처리 설계에서의 도전 과제

- 병렬 처리 아키텍처를 설계할 때, 추상화와 구현 사이의 격차를 메우는 것이 중요
 - 데이터 병렬성과 작업 병렬성: 데이터 흐름과 계산 흐름을 효과적으로 분배.
 - 메모리 접근 패턴 최적화: 데이터 지역성(locality)을 활용하여 메모리 대역폭 문제를 해결.
 - 리소스 스케줄링: 작업을 최적화하여 프로세서의 활용률을 극대화.

An example: Programming with ISPC

<https://ispc.github.io/index.html>

<https://github.com/ispc/ispc>

A great read: “The Story of ISPC” (by Matt Pharr)

- <https://pharr.org/matt/blog/2018/04/30/ispc-all.html>

- Go read it!

SPMD-Intel SIMD programming with ISPC

What is ISPC

- **Intel SPMD Program Compiler**

- **Single Program Multiple Data(SPMD)**

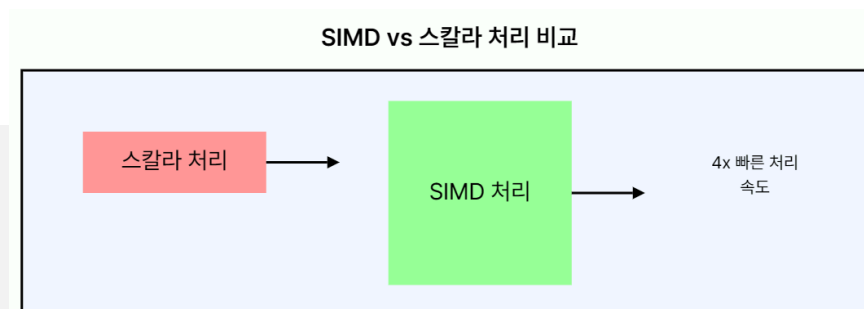
- 벡터 코드 작성을 위한 컴파일러 및 언어
- C 기반 언어
- 사용이 간편하고 기존 코드에 통합 가능
- **It is not a Auto-Vectorizing compiler**
 - 벡터는 타입 시스템에서 명시적으로 선언
 - 개발자가 스칼라와 벡터를 명시적으로 선언

ISPC(SIMD)의 기본 원리 이해하기

- SIMD의 핵심 아이디어는 단순.
- 하나의 명령어로 여러 개의 데이터를 동시에 처리하는 것.
- 이는 전통적인 스칼라 처리 방식과는 다름.

스칼라 vs SIMD 처리 비교:

- 스칼라 처리: $A + B = C$ (한 번에 하나의 연산)
- SIMD 처리: $[A1, A2, A3, A4] + [B1, B2, B3, B4] = [C1, C2, C3, C4]$ (한 번에 여러 연산)



- SIMD를 사용하면 데이터 병렬성(Data Parallelism)을 활용할 수 있음.
- 이는 같은 연산을 여러 데이터에 동시에 적용할 때 특히 유용.
- 예를 들어, 이미지 처리나 물리 시뮬레이션과 같은 작업에서 SIMD는 놀라운 성능 향상을 가져올 수 있음

ISPC(SIMD)의 기본 원리 이해하기

- SIMD 명령어는 특별한 레지스터를 사용.
- 이 레지스터는 여러 개의 데이터 요소를 동시에 저장 가능.
 - 예를 들어, 128비트 SIMD 레지스터는 4개의 32비트 부동 소수점 숫자를 동시에 저장하고 처리 가능

SIMD 장점

- **처리 속도 향상**: 여러 데이터를 동시에 처리하므로 전체적인 연산 속도 향상
- **에너지 효율성**: 같은 양의 데이터를 처리하는 데 필요한 전력 소비감소
- **코드 간소화**: 복잡한 루프 구조를 단순화할 수 있어 코드 가독성이 향상.

SIMD 단점(주의점)

- **데이터 정렬**: SIMD 연산은 대부분 정렬된 데이터에서 최적의 성능을 발휘함.
- **오버헤드**: 데이터를 SIMD 레지스터로 로드하고 다시 메모리로 저장하는 과정에서 약간의 오버헤드가 발생할 수 있음.
- **프로그래밍 복잡성**: SIMD 프로그래밍은 일반적인 스칼라 프로그래밍보다 복잡할 수 있음.

C 언어에서의 ISPC(SIMD) 구현

- C 언어에서 SIMD를 구현하는 방법은 크게 세 가지가 있음

1. 인트린식(Intrinsics) 사용: 컴파일러가 제공하는 특별한 함수를 사용하여 SIMD 명령어를 직접 호출.
2. 어셈블리 코드 삽입: 인라인 어셈블리를 사용하여 SIMD 명령어를 직접 작성.
3. 자동 벡터화: 컴파일러의 최적화 기능을 활용하여 자동으로 SIMD 명령어를 생성.

➤ 이 중에서 가장 일반적으로 사용되는 방법은 인트린식을 사용함.

➤ 인트린식은 어셈블리 코드를 직접 작성하는 것보다 쉽고, 자동 벡터화보다 더 세밀한 제어가 가능함.

📌 SIMD 인트린식을 사용하려면 해당 헤더 파일을 포함해야 함.

📌 예를 들어, SSE 인트린식을 사용하려면 `#include <xmmintrin.h>`를 추가해야 함

C 언어에서의 ISPC(SIMD) 구현

- SSE(Streaming SIMD Extensions)를 사용한 간단한 벡터 덧셈 예제

```
#include <xmmintrin.h>

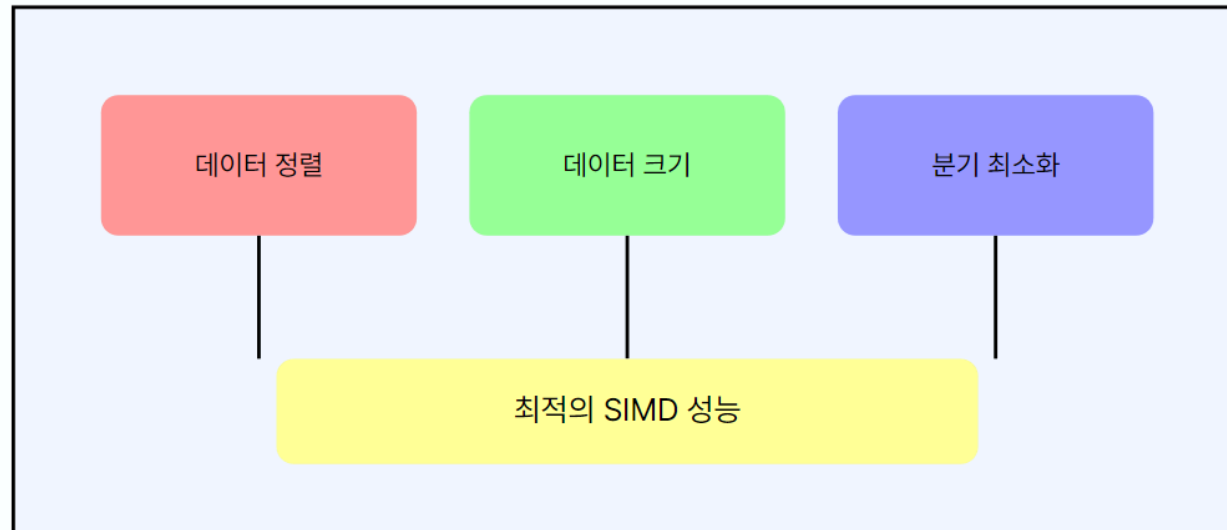
void vector_add_simd(float *a, float *b, float *result, int size) {
    for (int i = 0; i < size; i += 4) {
        __m128 va = _mm_load_ps(&a[i]);
        __m128 vb = _mm_load_ps(&b[i]);
        __m128 vr = _mm_add_ps(va, vb);
        _mm_store_ps(&result[i], vr);
    }
}
```

- `__m128`은 4개의 단정밀도 부동 소수점 숫자를 저장할 수 있는 128비트 SIMD 레지스터타입.
- `_mm_load_ps`, `_mm_add_ps`, `_mm_store_ps`는 SSE 인트린식 함수로, 각각 메모리에서 데이터를 SIMD 레지스터로 로드하고, SIMD 덧셈을 수행하고, 결과를 메모리에 저장하는 역할을 함.

C 언어에서의 ISPC(SIMD) 구현

- SIMD를 효과적으로 사용하기 위해서는 다음과 같은 점들을 고려
 - 데이터 정렬: SIMD 연산은 대부분 정렬된 데이터에서 최적의 성능을 발휘.
 - ➔ 16바이트 경계에 정렬된 데이터를 사용하는 것이 좋음.
 - 데이터 크기: SIMD 레지스터의 크기에 맞는 데이터 크기를 사용하는 것이 효율적임.
 - ➔ 예를 들어, SSE는 128비트 레지스터를 사용하므로 4개의 32비트 부동 소수점 숫자를 한 번에 처리하기에 적합
 - 분기 최소화: SIMD 연산은 모든 데이터 요소에 대해 동일한 연산을 수행할 때 가장 효과적임.
 - ➔ 조건문이나 분기가 많은 코드는 SIMD 최적화에 적합하지 않을 수 있음.

SIMD 최적화 고려사항



SIMD 명령어 세트 소개

- SIMD 명령어 세트는 프로세서 제조업체와 아키텍처에 따라 다양
- 가장 널리 사용되는 SIMD 명령어 세트들을 살펴보자

1.SSE (Streaming SIMD Extensions): Intel에서 개발한 SIMD 명령어 세트로, x86 아키텍처에서 사용

→. SSE, SSE2, SSE3, SSSE3, SSE4 등의 버전이 있음.

2.AVX (Advanced Vector Extensions): SSE의 후속 버전으로, 더 넓은 벡터 레지스터(256비트)를 지원

→ AVX, AVX2, AVX-512 등의 버전이 있음.

3.NEON: ARM 프로세서에서 사용되는 SIMD 명령어 세트.

- 각 SIMD 명령어 세트는 고유한 특징과 장단점을 가지고 있음.
 - 예를 들어, AVX는 SSE보다 더 넓은 레지스터를 사용하여 한 번에 더 많은 데이터를 처리할 수 있지만, 더 높은 전력을 소비.

SIMD 명령어 세트 소개

- SIMD 명령어 세트는 프로세서 제조업체와 아키텍처에 따라 다양
- 가장 널리 사용되는 SIMD 명령어 세트들을 살펴보자

1.SSE (Streaming SIMD Extensions): Intel에서 개발한 SIMD 명령어 세트로, x86 아키텍처에서 사용

→. SSE, SSE2, SSE3, SSSE3, SSE4 등의 버전이 있음.

2.AVX (Advanced Vector Extensions): SSE의 후속 버전으로, 더 넓은 벡터 레지스터(256비트)를 지원

→ AVX, AVX2, AVX-512 등의 버전이 있음.

3.NEON: ARM 프로세서에서 사용되는 SIMD 명령어 세트.

4.AltiVec: PowerPC 프로세서에서 사용되는 SIMD 명령어 세트.

- 각 SIMD 명령어 세트는 고유한 특징과 장단점을 가지고 있음.

→ 예를 들어, AVX는 SSE보다 더 넓은 레지스터를 사용하여 한 번에 더 많은 데이터를 처리할 수 있지만, 더 높은 전력을 소비.

 **팁:** 플랫폼을 지원해야 하는 경우, 런타임에 사용 가능한 SIMD 명령어 세트를 확인하고 그에 맞는 코드를 실행하는 방식 (CPU 디스패치)을 사용할 수 있음.

SSE와 AVX를 사용한 간단한 벡터 곱셈 예제

```
// SSE 버전
#include <xmmintrin.h>

void vector_multiply_sse(float *a, float *b, float *result, int size) {
    for (int i = 0; i < size; i += 4) {
        __m128 va = _mm_load_ps(&a[i]);
        __m128 vb = _mm_load_ps(&b[i]);
        __m128 vr = _mm_mul_ps(va, vb);
        _mm_store_ps(&result[i], vr);
    }
}
```

```
// AVX 버전
#include <immintrin.h>

void vector_multiply_avx(float *a, float *b, float *result, int size) {
    for (int i = 0; i < size; i += 8) {
        __m256 va = _mm256_load_ps(&a[i]);
        __m256 vb = _mm256_load_ps(&b[i]);
        __m256 vr = _mm256_mul_ps(va, vb);
        _mm256_store_ps(&result[i], vr);
    }
}
```

AVX 버전은 한 번에 8개의 부동 소수점
숫자를 처리할 수 있어 더 높은 처리량을
제공함

SSE vs AVX 비교

SSE vs AVX 비교

SSE

128비트 레지스터

4개의 32비트 float

VS

AVX

256비트 레지스터

8개의 32비트 float

SIMD 명령어 세트를 선택할 때의 고려 사항

- 타겟 하드웨어: 개발 중인 애플리케이션이 실행될 하드웨어의 SIMD 지원 여부를 확인해야 함.
- 성능 요구사항: 더 넓은 SIMD 레지스터를 사용하면 일반적으로 더 높은 성능을 얻을 수 있지만, 모든 상황에서 그렇지 는 않음.
- 전력 효율성: 더 넓은 SIMD 레지스터를 사용하면 전력 소비가 증가할 수 있음. 모바일 디바이스와 같은 전력 제한이 있는 환경에서는 이를 고려해야 함.
- 코드 복잡성: 더 새로운 SIMD 명령어 세트를 사용하면 코드가 복잡해질 수 있음. 유지보수성과 성능 사이의 균형을 고려해야 함.

SIMD를 활용한 실제 최적화 사례 연구

이미지 처리 최적화

이미지 밝기 조정 연산을 SIMD로 최적화 예시

```
#include <immintrin.h>

void adjust_brightness_simd(unsigned char *image, int size, float factor) {
    __m256 vfactor = _mm256_set1_ps(factor);

    for (int i = 0; i < size; i += 32) { // 32 bytes = 256 bits
        __m256i vdata = _mm256_loadu_si256((__m256i*)&image[i]);
        __m256 vfloat = _mm256_cvtepi32_ps(_mm256_cvtepu8_epi32(_mm256_extracti128_si256(vdata, 0)));
        vfloat = _mm256_mul_ps(vfloat, vfactor);
        __m256i vresult = _mm256_cvtps_epi32(vfloat);
        vresult = _mm256_packus_epi32(vresult, vresult);
        vresult = _mm256_packus_epi16(vresult, vresult);
        _mm_storeu_si128((__m128i*)&image[i], _mm256_extracti128_si256(vresult, 0));
    }
}
```

이 코드는 AVX2 명령어를 사용하여 한 번에 32개의 픽셀 밝기를 조정

SIMD를 활용한 실제 최적화 사례 연구

물리 시뮬레이션 가속화

물리 엔진에서 SIMD를 사용하면 많은 수의 입자나 객체의 위치, 속도를 동시에 업데이트

```
#include <immintrin.h>

void update_positions_simd(float *positions, float *velocities, int count) {
    __m256 dt = _mm256_set1_ps(0.016f); // 시간 간격 (예: 60 FPS)

    for (int i = 0; i < count; i += 8) {
        __m256 pos = _mm256_loadu_ps(&positions[i]);
        __m256 vel = _mm256_loadu_ps(&velocities[i]);

        __m256 new_pos = _mm256_add_ps(pos, _mm256_mul_ps(vel, dt));

        _mm256_storeu_ps(&positions[i], new_pos);
    }
}
```

- 이 코드는 8개의 입자 위치를 동시에 업데이트
- 대규모 시뮬레이션에서 이러한 최적화는 큰 차이를 만들 수 있음

SIMD를 활용한 실제 최적화 사례 연구

오디오 처리 최적화

볼륨 조절을 SIMD로 구현

```
#include <immintrin.h>

void adjust_volume_simd(float *audio, int samples, float volume) {
    __m256 vvolume = _mm256_set1_ps(volume);

    for (int i = 0; i < samples; i += 8) {
        __m256 vdata = _mm256_loadu_ps(&audio[i]);
        __m256 vresult = _mm256_mul_ps(vdata, vvolume);
        _mm256_storeu_ps(&audio[i], vresult);
    }
}
```

- 이 코드는 8개의 오디오 샘플의 볼륨을 동시에 조정.
- 실시간 오디오 처리에서 이러한 최적화는 CPU 사용량을 크게 줄일 수 있음

SIMD를 활용한 실제 최적화 사례 연구

암호화 알고리즘 가속화

AES 암호화의 일부 연산을 SIMD로 최적화

```
#include <wmmintrin.h> // AES 인트린식을 위한 헤더

void aes_encrypt_round_simd(__m128i *state, __m128i round_key) {
    *state = _mm_aesenc_si128(*state, round_key);
}
```

- 이 코드는 AES 암호화의 한 라운드를 SIMD 명령어를 사용하여 수행
- 대량의 데이터를 암호화할 때 상당한 성능 향상을 기대할 수 있음

SIMD를 활용한 실제 최적화 사례 연구

SIMD 최적화 성능 향상 예시



실제 성능 향상 정도는 구체적인 사용 사례, 데이터 특성, 하드웨어 등 여러 요인에 따라 달라질 수 있음.

SIMD 프로그래밍의 도전과 해결 전략

1. 데이터 정렬 문제

도전:

- SIMD 연산은 대부분 정렬된 데이터에서 최적의 성능을 발휘,
- 정렬되지 않은 데이터를 사용하면 성능이 크게 저하될 수 있음

해결 전략:

- 메모리 할당 시 정렬된 메모리를 사용. (예: `aligned_alloc` 함수 사용)
- 데이터 구조를 SIMD 친화적으로 재구성. (예: AoS 대신 SoA 사용)
- 정렬되지 않은 데이터를 처리할 경우 `_mm_loadu_ps`와 같은 비정렬 로드 명령어를 사용 (예: `_mm_loadu_ps`)

```
// 정렬된 메모리 할당 예시
float* aligned_data = (float*)aligned_alloc(32, size * sizeof(float));

// SoA (Structure of Arrays) 예시
struct Particles {
    float* x;
    float* y;
    float* z;
} particles;

// 정렬되지 않은 로드 예시
__m256 data = _mm256_loadu_ps(unaligned_ptr);
```

SIMD 프로그래밍의 도전과 해결 전략

2. 분기 처리

도전:

- SIMD는 모든 데이터에 대해 동일한 연산을 수행할 때 가장 효율적
- 조건문이나 분기가 많은 코드는 SIMD 최적화에 적합하지 않을 수 있음

해결 전략:

- 마스크 연산을 사용하여 조건부 실행을 구현.
- 분기 예측이 가능한 경우, 분기 대신 산술 연산을 사용.
- 데이터를 사전에 정렬하여 분기를 최소화.

```
// 마스크 연산 예시
__m256 condition = _mm256_cmp_ps(data, _mm256_set1_ps(threshold),
_CMP_GT_OS);
__m256 result = _mm256_blendv_ps(value_if_false, value_if_true, condition);

// 분기 대신 산술 연산 사용 예시
__m256 abs_value = _mm256_max_ps(_mm256_sub_ps(_mm256_setzero_ps(),
value), value);
```

SIMD 프로그래밍의 도전과 해결 전략

3. 데이터 의존성

도전:

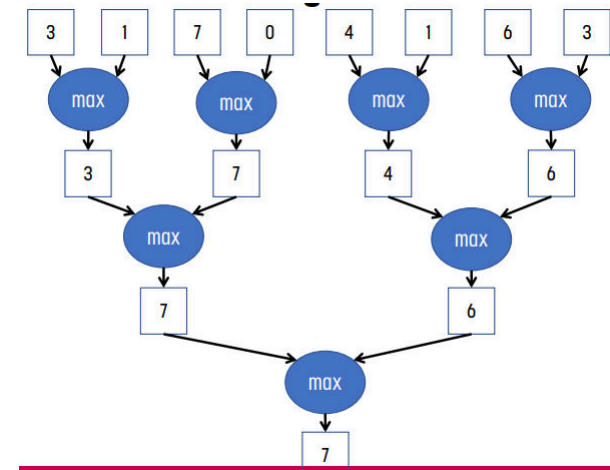
- SIMD 연산은 데이터 간 의존성이 없을 때 가장 효과적
- 연속된 연산 간에 데이터 의존성이 있으면 성능이 저하될 수 있음

해결 전략:

- 루프 언롤링을 사용하여 독립적인 연산을 늘림.
- 데이터 재구성을 통해 의존성을 줄임.
- 가능한 경우 병렬 리덕션 (parallel reduction) 알고리즘을 사용.

```
// 루프 언롤링 예시
for (int i = 0; i < size; i += 32) {
    __m256 sum1 = _mm256_load_ps(&data[i]);
    __m256 sum2 = _mm256_load_ps(&data[i + 8]);
    __m256 sum3 = _mm256_load_ps(&data[i + 16]);
    __m256 sum4 = _mm256_load_ps(&data[i + 24]);
    // 연산 수행
}

// 병렬 리덕션 예시
__m256 sum = _mm256_setzero_ps();
for (int i = 0; i < size; i += 8) {
    sum = _mm256_add_ps(sum, _mm256_load_ps(&data[i]));
}
float total = horizontal_sum(sum); // 256비트 벡터의 요소들을 모두 더하는 함수
```



Loop_unrolling (루프 풀기)

<https://blog.naver.com/acwboy/50083036889>

```
void normal_add(u_int *x, u_int*y,u_int size,
u_int *result)
{
    u_int sum=0;
    u_int i=0;
    for(i=0;i<size;i++)
    {
        sum += x[i] + y[i];
    }
    *result = sum;
}
```

```
void unroll_add(u_int *x, u_int* y, u_int size, u_int *result)
{
    u_int sum=0;
    u_int i=0;
    u_int ssize = (size / 8) * 8; //8번씩 할 것이므로 8의 배수로 만듦

    for(i=0;i<ssize;i+=8) //한번 루프에 8번씩 반복한다.
    {
        sum += x[i ] + y[i ];
        sum += x[i+1] + y[i+1];
        sum += x[i+2] + y[i+2];
        sum += x[i+3] + y[i+3];

        sum += x[i+4] + y[i+4];
        sum += x[i+5] + y[i+5];
        sum += x[i+6] + y[i+6];
        sum += x[i+7] + y[i+7];

    }

    for(i; i < size; i++) //나머지 부분
    {
        sum += x[i] + y[i];
    }
    *result = sum;
}
```

SIMD 프로그래밍의 도전과 해결 전략

4. 메모리 대역폭 제한

도전:

- SIMD 연산은 많은 데이터를 빠르게 처리할 수 있지만, 메모리 대역폭이 병목이 될 수 있음

해결 전략:

- 데이터 지역성을 최대화하여 캐시 사용을 최적화.
- 프리페치 명령어를 사용하여 데이터를 미리 캐시로 로드.
- 연산 강도를 높여 메모리 접근 대비 연산 비율을 개선

/ 프리페치 예시

```
_mm_prefetch((char*)&data[i + 64], _MM_HINT_T0);
```

// 연산 강도 높이기 예시

```
for (int i = 0; i < size; i += 8) {  
    __m256 a = _mm256_load_ps(&data_a[i]);  
    __m256 b = _mm256_load_ps(&data_b[i]);  
    __m256 c = _mm256_load_ps(&data_c[i]);  
    __m256 result = _mm256_fmadd_ps(a, b, c); // a * b + c  
    _mm256_store_ps(&output[i], result);  
}
```

SIMD 프로그래밍의 도전과 해결 전략

5. 이식성과 유지보수성

도전:

- SIMD 코드는 특정 명령어 세트에 의존적이며, 가독성이 떨어질 수 있음.
- 또한 다른 아키텍처로의 이식이 어려울 수 있음

해결 전략:

- SIMD 연산을 추상화하는 라이브러리나 매크로를 사용.
- 런타임에 사용 가능한 SIMD 기능을 감지하여 적절한 구현을 선택.
- SIMD 코드와 스칼라 코드를 병행 유지하여 이식성을 확보.

```
// SIMD 추상화 예시
#ifdef __AVX__
    #define SIMD_ADD(a, b) _mm256_add_ps(a, b)
#elif defined(__SSE__)
    #define SIMD_ADD(a, b) _mm_add_ps(a, b)
#else
    #define SIMD_ADD(a, b) ((a) + (b))
#endif
```

```
// 런타임 기능 감지 예시
if (cpu_supports_avx2()) {
    process_data_avx2(data, size);
} else if (cpu_supports_sse4_1()) {
    process_data_sse4_1(data, size);
} else {
    process_data_scalar(data, size);
}
```

SIMD 프로그래밍의 미래와 발전 방향

1. 더 넓은 SIMD 레지스터

SIMD 레지스터의 폭은 계속해서 넓어지고 있습니다. 예를 들어, AVX-512는 512비트 레지스터를 제공합니다. 이는 한 번의 연산으로 더 많은 데이터를 처리할 수 있게 해줍니다.

```
#include <immintrin.h>

void vector_add_avx512(float *a, float *b, float *result, int size)
{
    for (int i = 0; i < size; i += 16) { // 16 floats at a time
        __m512 va = _mm512_loadu_ps(&a[i]);
        __m512 vb = _mm512_loadu_ps(&b[i]);
        __m512 vr = _mm512_add_ps(va, vb);
        _mm512_storeu_ps(&result[i], vr);
    }
}
```

SIMD 프로그래밍의 미래와 발전 방향

2. 더 유연한 SIMD 연산

최신 SIMD 명령어 세트는 더 복잡하고 유연한 연산을 지원합니다. 예를 들어, AVX-512는 마스크 연산, 퍼뮤테이션, 압축/확장 연산 등을 제공합니다.

```
__m512 conditional_add(__m512 a, __m512 b, __mmask16  
mask) {  
    return _mm512_mask_add_ps(a, mask, a, b);  
}
```

SIMD 프로그래밍의 미래와 발전 방향

3. AI 및 머신 러닝을 위한 특화된 SIMD 명령어

AI와 머신 러닝의 중요성이 커짐에 따라, 이를 위한 특화된 SIMD 명령어들이 등장하고 있습니다. 예를 들어, Intel의 VNNI(Vector Neural Network Instructions)는 딥 러닝 추론을 가속화합니다.

```
// VNNI를 사용한 8비트 정수 행렬 곱셈
__m512i matrix_multiply_vnni(__m512i a, __m512i b) {
    return _mm512_dpbusds_epi32(_mm512_setzero_si512(), a,
b);
```

SIMD 프로그래밍의 미래와 발전 방향

4. 이기종 컴퓨팅과의 통합

SIMD는 CPU에서의 병렬 처리를 담당하지만, GPU, FPGA, 전용 AI 가속기 등 다양한 이기종 컴퓨팅 환경과의 통합이 중요해지고 있습니다. 이를 위한 통합 프로그래밍 모델과 도구들이 발전하고 있습니다.

```
// OpenCL을 사용한 이기종 컴퓨팅 예시
cl_kernel kernel = clCreateKernel(program, "vector_add", NULL);
clSetKernelArg(kernel, 0, sizeof(cl_mem), &buffer_A);
clSetKernelArg(kernel, 1, sizeof(cl_mem), &buffer_B);
clSetKernelArg(kernel, 2, sizeof(cl_mem), &buffer_C);
clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_size,
&local_size, 0, NULL, NULL);
```

SIMD 프로그래밍의 미래와 발전 방향

5. 자동 벡터화의 개선

컴파일러의 자동 벡터화 기능이 계속해서 개선되고 있습니다. 이는 개발자가 직접 SIMD 코드를 작성하지 않아도 효율적인 SIMD 코드를 생성할 수 있게 해줍니다.

```
// 컴파일러가 자동으로 벡터화할 수 있는 코드 예시
void auto_vectorized_add(float *a, float *b, float *result, int size) {
    #pragma omp simd
    for (int i = 0; i < size; i++) {
        result[i] = a[i] + b[i];
    }
}
```

SIMD 프로그래밍의 미래와 발전 방향

6. 에너지 효율성 향상

SIMD 연산의 에너지 효율성이 더욱 중요해지고 있습니다. 특히 모바일 및 임베디드 시스템에서 SIMD를 효율적으로 사용하기 위한 기술들이 발전하고 있습니다.

```
// ARM NEON을 사용한 저전력 SIMD 연산 예시
#include <arm_neon.h>

void energy_efficient_add(float32_t *a, float32_t *b, float32_t
*result, int size) {
    for (int i = 0; i < size; i += 4) {
        float32x4_t va = vld1q_f32(&a[i]);
        float32x4_t vb = vld1q_f32(&b[i]);
        float32x4_t vr = vaddq_f32(va, vb);
        vst1q_f32(&result[i], vr);
    }
}
```

Why ISPC?

● 병렬 처리 활용은 성능에 매우 중요

- 작업 병렬 처리(Task parallelism)
- SIMD 병렬 처리(SIMD parallelism)

● 내장함수(Intrinsics)로 수행 가능

- 내장함수(Intrinsics)는 플랫폼에 따라 다름.
- 이해하고 올바르게 적용하기 어려움

● 인텔 ISPC

- "닌자" 프로그래머에게 구걸하지 않고도 플롭을 더 쉽게 얻을 수 있음
- 높은 수준의 프로그래밍(개발 및 유지 관리가 더 쉬움)

Instruction Set

- ☐ MMX
- ☐ SSE family
- ☐ AVX family
- ☐ AVX-512 family
- ☐ AMX family
- ☐ SVMML
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Elementary Math Functions
- ☐ General Support
- ☐ Load
- ☐ Logical
- ☐ Mask
- ☐ Miscellaneous
- ☐ Move
- ☐ OS-Targeted
- ☐ Probability/Statistics
- ☐ Random
- ☐ Set
- ☐ Shift
- ☐ Special Math Functions
- ☐ Store
- ☐ String Compare
- ☐ Swizzle
- ☐ Trigonometry
- ☐ Unknown

ISPC Language

- C Based
- Code looks sequential but is parallel
- Two new keywords:
 - **uniform** : scalar value
 - **varying** : vector value
- *foreach* vector iteration

```
export void rgb2grey(uniform int N,  
                    uniform float R[],  
                    uniform float G[],  
                    uniform float B[],  
                    uniform float grey[]) {  
    foreach( i = 0 ... N) {  
        varying float vGray = 0.3f * R[i] + 0.59f * G[i]  
                                + 0.11f * B[i];  
  
        grey[i] = vGray;  
    }  
}
```

ISPC Integration

- ISPC 컴파일러는 C/C++ 코드와의 통합에 필요한 모든 것을 생성.
- C/C++ 헤더 파일
 - 작성한 각 커널에 대한 API/함수 호출을 포함.
 - ISPC 커널에 정의된 모든 데이터 구조 포함
- 링크할 오브젝트 파일
- 런타임 또는 장황한 API 없음
 - 예를 들어 OpenCL의 경우

Good because

- 프로그래머는 더 이상 기반이 되는 ISA를 알 필요가 없음.
- 개발 및 유지 관리 비용 절감
- 최적화 도달 범위 증가
- 성능 향상

ISA(ISA (Instruction Set Architecture)):

하드웨어와 소프트웨어 사이의 Interface를 정의하는 것.

하드웨어와 프로그램 사이의 매개체 역할을 하는 것

Vector Loops

foreach

```
export uniform int sum (uniform int val[], uniform int N) {  
    varying int sum = 0;  
    foreach( i = 0 ... N) {  
        sum += val[i];  
    }  
    return reduce_add(sum);  
}
```

foreach

- 루프용 SIMD
- SIMD 폭의 청크(Set of Data)에 대해 반복.
- 모든 SIMD 레인을 위한 마스크되지 않은 Main Body
- 일부 SIMD 레인이 비활성화되었을 때를 위한 마스크된 Tail Body
- foreach는 N 차원이 될 수 있으며, 각 차원은 varying

for loop

```
export uniform int sum(uniform int val[],uniform int N) {  
    varying int sum = 0;  
    for(varying int i = programIndex; i < N;  
        i+= programCount) {  
        sum += val[i];  
    }  
    return reduce_add(sum);  
}
```

For loop

- varying 인덱스인 for 루프 는 루프 본문에서 마스크를 사용
- Uniform인덱스인 for 루프는 마스크가 없음.

Vector Loops

foreach_active

- 각 활성 SIMD 레인을 직렬화(Serializes).
- 다양한 용도로 사용:
 - Atomic operations
 - Custom reductions
 - Calls to uniform functions

```
export uniform int sum(uniform int val[], uniform int N) {  
    uniform int sum[programCount];  
    sum[programIndex] = 0;  
    for(varying int i = programIndex; i < N; i += programCount)  
    {  
        sum[programIndex] += val[i];  
    }  
    uniform int ret = 0;  
    foreach_active(j) {  
        ret += sum[j];  
    }  
}
```

Recall: example program from last class

Compute $\sin(x)$ using Taylor expansion: $\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \dots$
for each element of an array of N floating-point numbers

```
void sinx(int N, int terms, float* x, float* result)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

Invoking sinx()

C++ code: main.cpp

```
#include "sinx.h"

int main(int argc, void** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];

    // initialize x here

    sinx(N, terms, x, result);

    return 0;
}
```

순차처리

순차처리

main()

sinx()

Call to sinx()
Control transferred to sinx() func

Return from sinx()
Control transferred back to main()

C++ code: sinx.cpp

```
void sinx(int N, int terms, float* x, float* result)
{
    for (int i=0; i<N; i++)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        int denom = 6; // 3!
        int sign = -1;

        for (int j=1; j<=terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }

        result[i] = value;
    }
}
```

Invoking sinx() in ISPC

핵심은 ISPC가 병렬 처리를 위해 실행 모델을 어떻게 변경하는 가

C++ code: main.cpp

```
#include "sinx_ispc.h"

int main(int argc, void** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];

    // initialize x here

    // execute ISPC code
    ispc_sinx(N, terms, x, result);
    return 0;
}
```

SPMD 프로그래밍 추상화

SPMD programming abstraction:

Call to ISPC function spawns "gang" of ISPC "program instances"

All instances run ISPC code concurrently

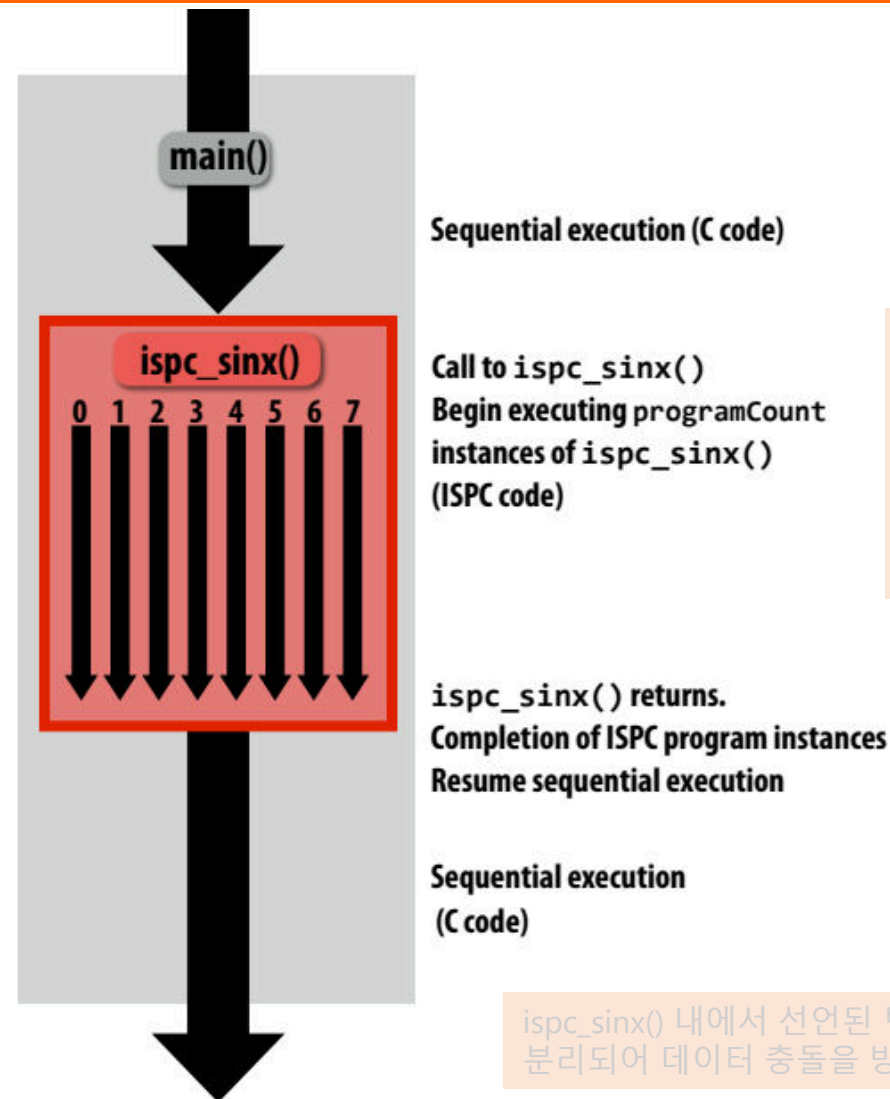
Each instance has its own copy of local variables

Upon return, all instances have completed

In this illustration programCount = 8



- ISPC에서 sinx() 함수는 SPMD 프로그래밍 모델로 작성되며, 호출 시 여러 "Gang"의 ISPC 프로그램 인스턴스를 동시에 생성.
- 각 Gang은 서로 다른 데이터를 병렬로 처리하며 다음과 같은 특징을 가짐
 - 프로그램 인스턴스: 모든 Gang 인스턴스는 동일한 ISPC 코드를 실행하되, 각 인스턴스는 고유한 데이터의 일부를 처리
 - 고유한 지역(고컬) 변수들: 각 인스턴스는 자체 지역 변수에 대한 복사본을 가지므로 독립적인 계산이 가능



여기서의 'gang'은 모두 같은 작업을 수행하지만 서로 다른 데이터를 다루는 작업자들로 구성된 팀이라고 생각하면 됨. 각 인스턴스는 ISPC 코드를 동시에(병렬로) 실행

ispc_sinx() 내에서 선언된 변수는 각 인스턴스마다 분리되어 데이터 충돌을 방지.

sinx() in ISPC

C++ code: main.cpp

```
#include "sinx_ispc.h"

int main(int argc, void** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];

    // initialize x here

    // execute ISPC code
    ispc_sinx(N, terms, x, result);
    return 0;
}
```

프로그램 인스턴스에 대한 배열 요소의 "인터리브" 할당

모든 인스턴스에서 동일한 값을 갖는 변수를 선언하는데 사용



ISPC code: sinx.ispc

```
export void ispc_sinx(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    // assumes N % programCount = 0
    for (uniform int i=0; i<N; i+=programCount)
    {
        int idx = i + programIndex;
        float value = x[idx];
        float number = x[idx] * x[idx] * x[idx];
        uniform int denom = 6; // 3!
        uniform int sign = -1;

        for (uniform int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[idx] * x[idx];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[idx] = value;
    }
}
```

- **programCount**: 동시에 실행 중인 인스턴스 수, gang의 인스턴스 수 (uniform value:)
- **programIndex**: gang에 있는 현재 인스턴스의 ID. (a non-uniform value: "varying")
- **uniform**: 유형 모드. 모든 인스턴스는 이 변수에 대해 동일한 값을 갖는다. 이 변수의 사용은 순전히 최적화를 위한 것. 정확성을 위해 필요하지 않음

추상화:

ISPC는 SPMD(단일 프로그램, 다중 데이터) 추상화를 사용. 즉, 하나의 프로그램 (`ispc_sinx()`)을 작성하지만 여러 데이터 요소에서 동시에 실행.

병렬성:

C++는 함수를 순차적으로 실행하는 반면, ISPC는 이러한 프로그램 인스턴스 'Gang(무리)' 을 생성하여 병렬성을 달성.

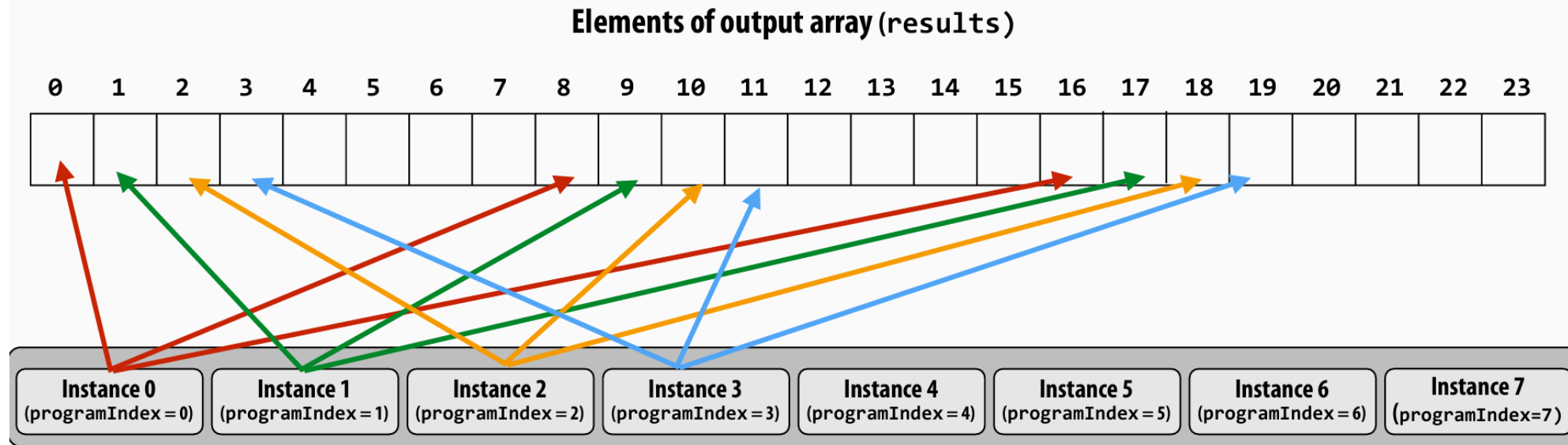
데이터 처리:

ISPC는 각 인스턴스에 고유한 데이터를 제공하고 공유 데이터에 대해 통일된 키워드를 제공함으로써 데이터 병렬 처리를 처리.

Interleaved assignment of program instances to loop iterations

ISPC 프로그램 인스턴스의 "Gang"

이 그림에서: Gang에는 8개의 인스턴스가 포함되어 있음: `programCount = 8`



ISPC에 루프(loop)가 있는 경우 해당 루프의 다른 반복을 **갱** 내의 다른 인스턴스에 할당하는 전략이 필요

- 인터리브 할당은 Deck(덱)에서 카드를 뽑는 것과 같음.
- 카드 덱(루프 반복을 나타냄)이 있고 플레이어 그룹(ISPC 프로그램 인스턴스를 나타냄)에게 카드를 나눠준다고 상상해보자.
- 각 플레이어에게 차례로 카드 한 장을 나눠주고 첫 번째 플레이어에게 돌아가는 식으로 진행.
- ISPC에서는 인터리브 할당을 통해 루프 반복이 라운드 로빈 방식으로 프로그램 인스턴스에 할당. 즉, 인스턴스 0은 반복 0, 인스턴스 1은 반복 1, 인스턴스 2는 반복 2를 받는 식. 마지막 인스턴스에 도달하면 인스턴스 0으로 돌아가서 다음 반복을 할당.
- 그럼, 왜 인터리브 할당이 필요한가요?



- ISPC는 SIMD(단일 명령어, 다중 데이터) 처리를 효율적으로 사용할 수 있는 프로그램을 작성하도록 설계됨.
- ISPC에서는 "갱"이라고 하는 프로그램 인스턴스 그룹이 서로 다른 데이터에 대해 동일한 코드를 동시에 실행.

Interleaved assignment of program instances to loop iterations

효율적인 SIMD 로드:

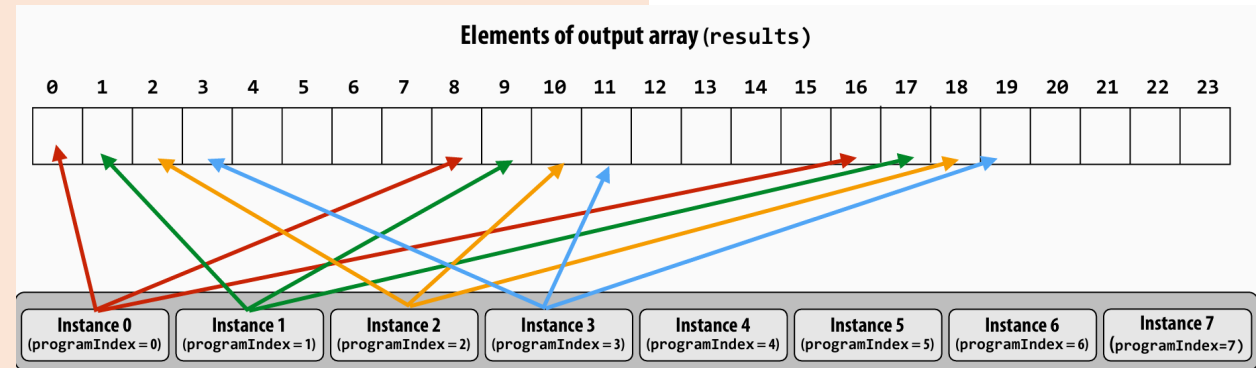
- 메모리의 데이터에 액세스할 때 인터리브 할당은 경우에 따라 더 효율적인 메모리 액세스 패턴으로 이어질 수 있음.
- 데이터가 메모리에 순차적으로 배치되어 있는 경우, 하나의 "패킹된 벡터 로드" 명령어로 여러 인스턴스의 데이터를 한 번에 가져올 수 있음. 이는 SIMD의 핵심 최적화 기능.

예시:

ISPC 인스턴스 8개(programCount = 8)와 루프 반복이 16개 있다고 가정. 할당은 다음과 같습니다:

인스턴스 0: 반복 0, 8
인스턴스 1: 반복 1, 9
인스턴스 2: 반복 2, 10
인스턴스 3: 반복 3, 11
인스턴스 4: 반복 4, 12
인스턴스 5: 반복 5, 13
인스턴스 6: 반복 6, 14
인스턴스 7: 반복 7, 15

왜 유리하죠??



요약

인터리브 할당은 ISPC 프로그램 인스턴스 간에 루프 반복을 분산하는 방법.

메모리에서 순차적 데이터에 액세스할 때 효율적인 "패킹 벡터 로드"를 가능하게 하므로 SIMD 처리에 유리할 수 있음

ISPC implements the gang abstraction using SIMD instructions

C++ code: main.cpp

```
#include "sinx_ispc.h"

int main(int argc, void** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];

    // initialize x here

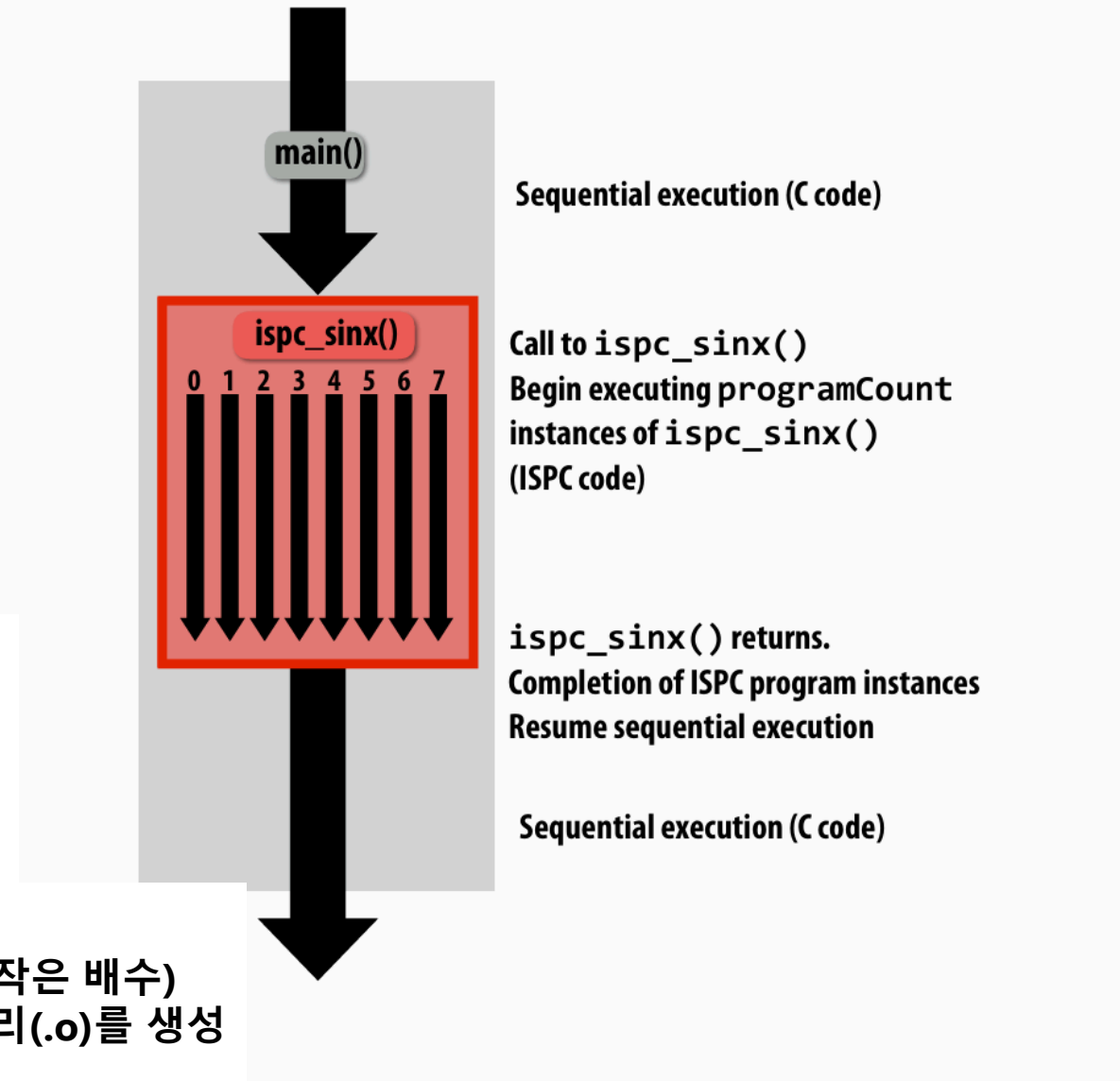
    // execute ISPC code
    ispc_sinx(N, terms, x, result);
    return 0;
}
```

SPMD programming abstraction:

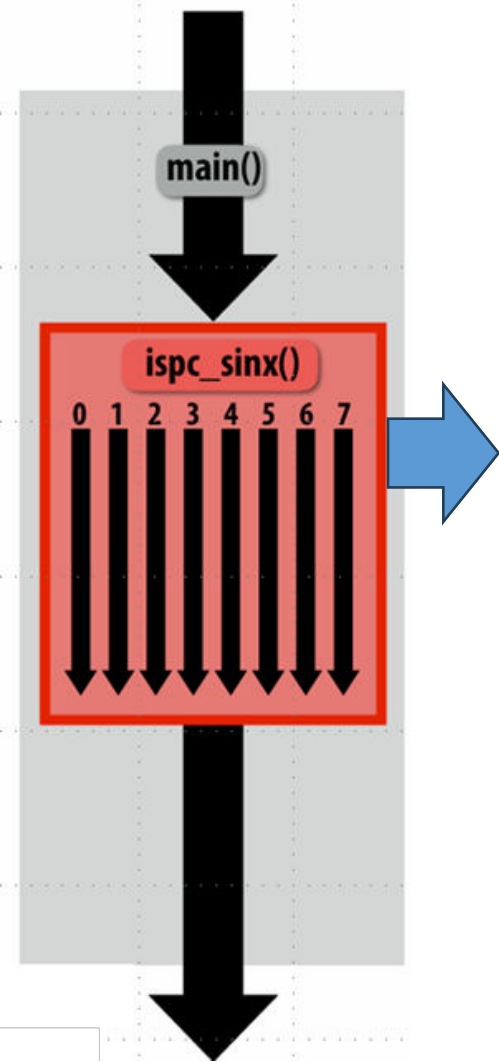
- ISPC 함수를 호출하면 ISPC "program instances"의 "Gang"이 스폰.
- 모든 인스턴스가 동시에 ISPC 코드를 실행.
- 반환 시 모든 인스턴스가 완료됨

ISPC 컴파일러가 SIMD 구현을 생성합니다:

- 갱의 인스턴스 수는 하드웨어의 SIMD 폭(또는 SIMD 폭의 작은 배수)
- ISPC 컴파일러는 SIMD 명령어가 포함된 C++ 함수 바이너리(.o)를 생성
- 생성된 객체에 대한 C++ 코드 링크는 평소와 같음.



ISPC implements the gang abstraction using SIMD instructions



- ISPC는 SIMD 명령어를 사용하여 SPMD 추상화를 영리하게 구현.
 - ISPC 컴파일러는 ISPC 코드를 분석하고 저수준 코드(SIMD 명령어 포함)를 생성하여 사용자가 SPMD 프로그램에서 설명한 병렬 동작을 구현.
 - “갱”에 포함된 ISPC 인스턴스 수는 종종 대상 하드웨어의 SIMD 폭과 관련이 있음.
 - 이를 통해 컴파일러는 각 인스턴스를 SIMD 레인에 효율적으로 매핑할 수 있음.
 - 따라서 개념적으로 여러 인스턴스에서 실행되는 ISPC 코드를 작성할 때 컴파일된 코드는 실제로 SIMD 명령어를 사용하여 해당 작업을 병렬로 수행.
- ISPC 컴파일러는 ISPC 코드를 가져와 일반 객체 파일(Linux의 .o 파일과 같은)을 생성.
 - 이 오브젝트 파일에는 필요한 SIMD 명령어가 포함되어 있음.
 - 다른 컴파일된 코드와 마찬가지로 메인 C++ 코드가 이 오브젝트 파일에 링크할 수 있음.
 - 이를 통해 C++ 코드에서 ISPC 함수를 호출.

간단히 설명하면 이렇게 생각해 보세요: “이 연산을 8번 병렬로 수행하고 싶습니다.”라고 ISPC에 말합니다. ISPC는 8개의 개별 워커를 생성하지 않습니다. 대신 SIMD를 사용하여 CPU에 “이 8개의 데이터에 대해 동시에 이 연산을 수행하라”고 지시합니다. 이것이 바로 ISPC가 저수준 하드웨어 기능(SIMD)을 사용하여 효율적으로 실행되는 고수준 추상화(SPMD)를 제공하는 방식입니다.

sinx() in ISPC: version 2

인스턴스에 대한 반복의 “덩어리(Blocked)” 할당이라는 개념이 도입

C++ code: main.cpp

```
#include "sinx_ispc.h"

int main(int argc, void** argv) {
    int N = 1024;
    int terms = 5;
    float* x = new float[N];
    float* result = new float[N];

    // initialize x here

    // execute ISPC code
    ispc_sinx_v2(N, terms, x, result);
    return 0;
}
```

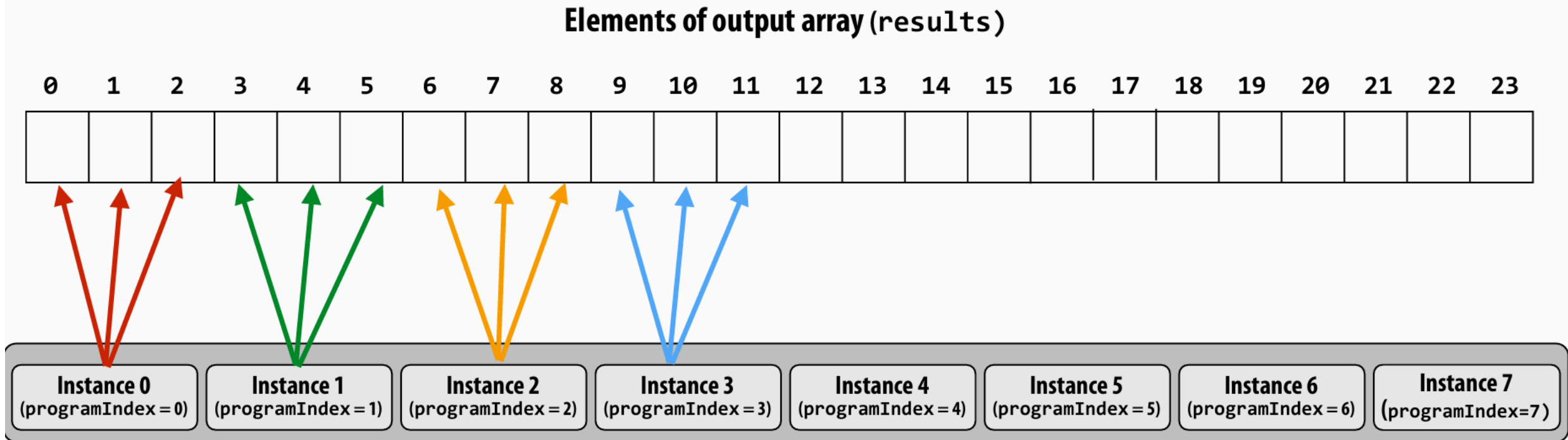
ISPC code: sinx.ispc

```
export void ispc_sinx_v2(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    // assume N % programCount = 0
    uniform int count = N / programCount;
    int start = programIndex * count;
    for (uniform int i=0; i<count; i++)
    {
        int idx = start + i;
        float value = x[idx];
        float number = x[idx] * x[idx] * x[idx];
        uniform int denom = 6; // 3!
        uniform int sign = -1;

        for (uniform int j=1; j<=terms; j++)
        {
            value += sign * number / denom;
            number *= x[idx] * x[idx];
            denom *= (j+3) * (j+4);
            sign *= -1;
        }
        result[idx] = value;
    }
}
```

- ◆ 각 인스턴스가 입력 배열의 흩어진 요소를 처리하는 대신 이제 각 인스턴스는 연속된 요소 덩어리(또는 “블록”)를 처리.
- $\text{uniform int count} = N / \text{programCount}$; 각 인스턴스가 처리할 요소 수를 계산. (이 **count** 변수가 모든 프로그램 인스턴스에서 동일한 값을 가진다는 것을 의미)
- 총 요소 수(N)를 인스턴스 수(programCount)로 균등하게 나눌 수 있다고 가정.
- $\text{int start} = \text{programIndex} * \text{count}$; 각 인스턴스의 시작 인덱스를 결정. (각 인스턴스가 x 배열에서 처리할 데이터의 시작 인덱스를 계산)
 - 예를 들어, programCount가 4이고 count가 256일 경우:
 - ✓ 인스턴스 0 (programIndex = 0) → 시작 인덱스: 0
 - ✓ 인스턴스 1 (programIndex = 1) → 시작 인덱스: 256
 - ✓ 인스턴스 2 (programIndex = 2) → 시작 인덱스: 512
 - ✓ 인스턴스 3 (programIndex = 3) → 시작 인덱스: 768
- ◆ programIndex는 각 ISPC 인스턴스의 고유 ID.
- ◆ (균일 $\text{int } i=0; i<\text{count}; i++$)에 대한 루프는 각 인스턴스가 할당된 데이터 블록에 대해서만 반복하도록 함.
 - 이것은 각 인스턴스가 자신의 블록을 처리하는 주 반복문. 각 인스턴스는 count만큼 반복하며 할당된 데이터 블록을 처리

Blocked assignment of program instances to loop iterations



In this illustration: gang contains eight instances: programCount = 8

- 배열이 있고 이를 동일한 크기의 세그먼트로 나눈다고 가정.
- 각 ISPC 인스턴스는 이러한 세그먼트 중 하나를 처리할 책임이 있음.
- 이는 인스턴스가 배열 전체에 흩어져 있는 요소를 처리하는 “인터리브” 할당과 대조적임..

Blocked assignment of program instances to loop iterations

블록 할당의 장점

- 지역성(Locality):
 - 블록 할당은 참조 지역성을 향상시킬 수 있음.
 - ➔ 이는 각 인스턴스가 메모리 상에서 가까운 위치에 있는 데이터를 접근하게 된다는 것을 의미하고
 - ➔ 이는 성능 면에서 유리함, 현대 프로세서는 자주 접근하는 데이터를 캐시에 저장하여 처리 속도를 높이기 때문. 즉, 인접한 데이터를 접근하면 이미 캐시에 있는 경우가 많아 더 빠르게 처리 가능.
- 간단한 논리 구성:
 - 경우에 따라 블록 할당은 데이터 의존성을 파악하거나 **경쟁 조건(race condition)**을 피하는 데 더 간단한 논리를 제공함.
즉, 여러 인스턴스가 동시에 동일한 데이터를 접근하거나 수정하려는 상황을 방지하기 쉬워짐.

트레이드오프(Trade-offs)

블록 할당은 SIMD 벡터 로드와 같은 경우에 최적이지 아닐 수 있음.
특히, 연속적이지 않은(non-contiguous) 방식으로 데이터를 접근해야 할 경우에는 성능상 불리할 수 있음.

예로, 어떤 것이 있을 가요

Schedule: interleaved assignment

"Gang" of ISPC program instances

Gang contains four instances: programCount = 8

	Instance 0 (programIndex = 0)	Instance 1 (programIndex = 1)	Instance 2 (programIndex = 2)	Instance 3 (programIndex = 3)	Instance 4 (programIndex = 4)	Instance 5 (programIndex = 5)	Instance 6 (programIndex = 6)	Instance 7 (programIndex = 7)
time								
i=0	0	1	2	3	4	5	6	7
i=1	8	9	10	11	12	13	14	15
i=2	16	17	18	19	20	21	22	23
i=3	24	25	26	27	28	29	30	31

A single "packed vector load" instruction (`vmovaps *`) efficiently implements:
`float value = x[idx];`
for all program instances, since the eight values are contiguous in memory

* see `_mm256_load_ps()` intrinsic function

```
...  
// assumes N % programCount = 0  
for (uniform int i=0; i<N; i+=programCount)  
{  
    int idx = i + programIndex;  
    float value = x[idx];  
}  
...
```

Schedule: interleaved assignment

C++

```
#include <immintrin.h> // AVX/AVX2 명령어 사용을 위한 헤더 파일

int main() {
    float data[8] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f};
    __m256 vector = _mm256_loadu_ps(data); // Vector Load 연산

    // 벡터 레지스터에 저장된 데이터 출력
    float result[8];
    _mm256_storeu_ps(result, vector);
    for (int i = 0; i < 8; i++) {
        printf("%f ", result[i]);
    }
    printf("\n");

    return 0;
}
```

`_mm256_loadu_ps()` 함수는 메모리 `data`에서 8개의 `float` 데이터를 읽어와 256비트 벡터 레지스터 `vector`에 저장.

Schedule: blocked assignment

“Gang” of ISPC program instances

Gang contains four instances: programCount = 8

	Instance 0 (programIndex = 0)	Instance 1 (programIndex = 1)	Instance 2 (programIndex = 2)	Instance 3 (programIndex = 3)	Instance 4 (programIndex = 4)	Instance 5 (programIndex = 5)	Instance 6 (programIndex = 6)	Instance 7 (programIndex = 7)
time								
i=0	0	8	16	24	32	40	48	56
i=1	1	9	17	25	33	41	49	57
i=2	2	10	18	26	34	42	50	58
i=3	3	11	19	27	35	43	51	59

float value = x[idx];
For all program instances now touches eight non-contiguous values in memory. Need “gather” instruction (vgatherdps *) to implement (gather is a more complex, and more costly SIMD instruction...)

```
uniform int count = N / programCount;  
int start = programIndex * count;  
for (uniform int i=0; i<count; i++) {  
    int idx = start + i;  
    float value = x[idx];  
    ...  
}
```

* see `_mm256_i32gather_ps()` intrinsic function

Raising level of abstraction with foreach

foreach 구문을 사용하여 병렬 연산의 추상화 수준을 높이는 방법

C++ code: main.cpp

```
#include "sinx_ispc.h"

int N = 1024;
int terms = 5;
float* x = new float[N];
float* result = new float[N];

// initialize x here

// execute ISPC code
sinx(N, terms, x, result);
```

foreach 구문의 핵심 개념

- 반복 작업의 추상화:
 - foreach 구문을 사용하면 반복적인 작업을 직접 루프를 작성하는 대신, 더 높은 수준에서 표현가능.
 - 이를 통해 코드의 가독성과 유지 보수성을 향상시킴
- 병렬 처리의 간편화:
 - ISPC의 foreach는 자동으로 병렬 처리를 수행.
 - 개발자는 병렬 처리에 대한 세부 사항을 직접 관리할 필요 없이, 작업의 논리에 집중할 수 있음.
- 코드의 간결성:
 - foreach 구문을 사용하면 반복적인 작업을 몇 줄의 코드로 표현할 수 있음.
 - 이는 코드의 길이를 줄이고, 오류 발생 가능성을 낮춤

ISPC code: sinx.ispc

```
export void ispc_sinx(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    foreach (i = 0 ... N)
    {
        float value = x[i];
        float number = x[i] * x[i] * x[i];
        uniform int denom = 6; // 3!
        uniform int sign = -1;

        for (uniform int j=1; j<=terms; j++)
        {
            value += sign * number / denom
            number *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

How might foreach be implemented?

Code written using foreach abstraction:

```
foreach (i = 0 ... N)
{
    // do work for iteration i here...
}
```

Implementation 1: program instance 0 executes all iterations

```
if (programCount == 0) {
    for (int i=0; i<N; i++) {
        // do work for iteration i here...
    }
}
```

Implementation 2: interleave iterations onto program instances

```
// assume N % programCount = 0
for (uniform int loop_i=0; loop_i<N; loop_i+=programCount)
{
    int i = loop_i + programIndex;
    // do work for iteration i here...
}
```

Implementation 3: block iterations onto program instances

```
// assume N % programCount = 0
uniform int count = N / programCount;
int start = programIndex * count;
for (uniform int loop_i=0; loop_i<count; loop_i++)
{
    int i = start + loop_i;
    // do work for iteration i here...
}
```

Implementation 4: dynamic assignment of iterations to instances

```
uniform int nextIter;
if (programCount == 0)
    nextIter = 0;

int i = atomic_add_local(&nextIter, 1);
while (i < N) {

    // do work for iteration i here...

    i = atomic_add_local(&nextIter, 1);
}
```

How might foreach be implemented?

- **foreach**는 개발자에게는 간단한 추상화를 제공하지만, 컴파일러는 이를 효율적인 병렬 코드로 변환해야 함.
- **foreach 구현의 핵심 고려 사항**
 - 병렬 처리 모델: ISPC는 SPMD(Single Program Multiple Data) 모델을 사용하므로, **foreach**는 각 **instance**에서 실행될 코드를 생성해야 함
 - 작업 분할 및 할당: **foreach**로 지정된 작업 범위를 **instance**들에 균등하게 분할하고 할당해야 함.
 - 메모리 접근 패턴 최적화: 병렬 처리 성능을 높이기 위해 메모리 접근 패턴을 최적화해야 함.
 - 제어 흐름 관리: **foreach** 내부의 제어 흐름(분기, 루프 등)을 **instance** 간에 동기화하고 관리해야 함.
- **구현 가능성**
 - 1.정적 스케줄링:
 - 컴파일 시간에 작업 범위를 **instance** 수에 따라 균등하게 분할.
 - 각 **instance**는 할당된 범위의 작업을 순차적으로 처리.
 - 장점: 간단하고 효율적.
 - 단점: 작업 부하가 균등하지 않은 경우 성능 저하가 발생할 수 있음.
 - 2.동적 스케줄링:
 - 실행 시간에 작업 범위를 작은 작업 단위로 나누고, 각 **instance**는 작업 단위가 완료되는 대로 다음 작업을 가져.
 - 장점: 작업 부하가 균등하지 않은 경우에도 높은 성능을 유지할 수 있음.
 - 단점: 동적 스케줄링을 위한 추가적인 오버헤드가 발생할 수 있음.
 - 3.인터리브 할당:
 - 작업 범위를 **instance** 수만큼 나누어 각 **instance**에 인터리브 방식으로 할당.
 - 장점: 메모리 접근 패턴을 최적화하여 캐시 효율성을 높일 수 있음.
 - 단점: 데이터 의존성이 있는 경우 구현이 복잡해질 수 있음.
 - 4.벡터화:
 - **foreach** 내부의 연산을 벡터 연산으로 변환하여 SIMD(Single Instruction, Multiple Data)를 활용.
 - 장점: 데이터 수준 병렬성을 극대화하여 높은 성능을 얻을 수 있음.
 - 단점: 모든 연산이 벡터화 가능한 것은 아님.

How might foreach be implemented?

- 컴파일러 최적화:

- ❖ 컴파일러는 개발자가 작성한 코드를 최대한 효율적인 기계어로 변환하는 역할.
- ❖ foreach 구문은 컴파일러 최적화의 좋은 예시.

- 실행 시간 오버헤드:

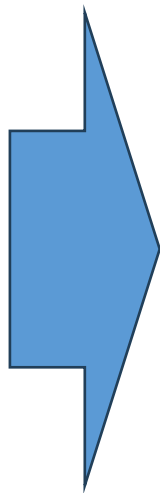
- ❖ 동적 스케줄링과 같은 고급 기술은 실행 시간 오버헤드를 발생시킬 수 있음.
- ❖ 컴파일러는 이러한 오버헤드를 최소화해야 함.

- 데이터 의존성:

- ❖ 데이터 의존성은 병렬 처리 성능을 제한하는 중요한 요소.
- ❖ 컴파일러는 데이터 의존성을 분석하고 병렬 처리 방식을 결정해야 함.

Thinking about iterations, not parallel execution

많은 간단한 경우, 프로그래머는 foreach를 사용하여 마치 순차적인 프로그램처럼 프로그램을 표현 가능



```
export void ispc_function(  
    uniform int    N,  
    uniform float* x,  
    uniform float* y)  
{  
    foreach (i = 0 ... N)  
    {  
        float val = x[i];  
        float result;  
  
        // do work here to compute  
        // result from val  
  
        y[i] = result;  
    }  
}
```

What does this program do?

```
// main C++ code:
const int N = 1024;
float* x = new float[N/2];
float* y = new float[N];

// initialize N/2 elements of x here

// call ISPC function
absolute_repeat(N/2, x, y);
```

```
// ISPC code:
export void absolute_repeat(
    uniform int N,
    uniform float* x,
    uniform float* y)
{
    foreach (i = 0 ... N)
    {
        if (x[i] < 0)
            y[2*i] = -x[i];
        else
            y[2*i] = x[i];
        y[2*i+1] = y[2*i];
    }
}
```

This ISPC program computes the absolute value of elements of x , then repeats it twice in the output array y

```
export void absolute_repeat(...):
```

- **absolute_repeat**라는 이름의 ISPC 함수를 선언
- **export** 키워드는 이 함수를 **C++ 코드에서 호출할 수 있도록** 만들어줌

```
uniform int N, uniform float* x, uniform float* y
```

- **함수의 매개변수(parameter)들**
 - **uniform int N:**
정수형 N 으로, **모든 ISPC 프로그램 인스턴스(각 SIMD 연산의 lane)**에서 동일한 값을 가짐.
 - **uniform float* x:**
실수형(float) 배열 x 를 가리키는 포인터. 이 포인터도 모든 인스턴스에서 동일함.
 - **uniform float* y:**
실수형 배열 y 를 가리키는 또 다른 포인터로, 역시 모든 인스턴스에서 동일함

What does this program do?

```
// main C++ code:
const int N = 1024;
float* x = new float[N];
float* y = new float[N];

// initialize N elements of x

// call ISPC function
shift_negative(N, x, y);
```

```
// ISPC code:
export void shift_negative(
    uniform int N,
    uniform float* x,
    uniform float* y)
{
    foreach (i = 0 ... N)
    {
        if (i >= 1 && x[i] < 0)
            y[i-1] = x[i];
        else
            y[i] = x[i];
    }
}
```

The output of this program is undefined!

Possible for multiple iterations of the loop
body to write to same memory location

병렬 코드에서 발생할 수 있는 **쓰기 충돌(write conflicts)**
의 위험성을 보여주는 **경고성 예제**

What does this program do?

이 프로그램은 입력 배열 x 의 음수 값을 왼쪽으로 한 칸 이동하여 배열 y 에 저장하려고 합니다.
하지만 중요한 점은, 이 프로그램의 출력은 ****정의되지 않은 결과(undefined behavior)****를 낼 수 있다는 것입니다.

코드 분석:

- `export void shift_negative(...)`
 - ✓ `shift_negative`라는 ISPC 함수를 선언합니다.
 - ✓ `export` 키워드는 이 함수가 **C++ 코드에서 호출 가능**하다는 의미.
- `uniform int N, uniform float* x, uniform float* y`
 - ✓ 함수의 매개변수들:
 - N : 배열의 크기
 - x : 입력 배열 포인터
 - y : 출력 배열 포인터모두 **모든 인스턴스에서 동일한 값을** 가집니다 (uniform).
- `foreach (i = 0 ... N)`
 - ✓ ISPC의 **병렬 루프**.
 - ✓ i 가 0부터 $N-1$ 까지 순회하며 각 반복은 병렬로 실행될 수 있음.
- `if (i >= 1 && x[i] < 0)`
 - ✓ 핵심 로직.
 - i 가 1 이상이고, $x[i]$ 값이 음수라면
 - $y[i-1] = x[i]$; → 음수 값을 왼쪽으로 한 칸 이동시킴.
 - 그렇지 않으면
 - $y[i] = x[i]$; → 값을 그대로 복사.

What does this program do?

1. 쓰기 충돌(Potential Write Conflicts)

cpp

```
if (i >= 1 && x[i] < 0)
    y[i-1] = x[i];
```

- 이 조건 때문에, 만약 $x[1]$ 과 $x[2]$ 가 모두 음수라면 다음과 같은 문제가 발생함:
 - $i = 1$ 인 인스턴스는 $y[0] = x[1]$ 을 시도.
 - $i = 2$ 인 인스턴스도 $y[1] = x[2]$ 가 아닌 $**y[1 - 1] = x[2]$, 즉 $y[0] = x[2]**$ 를 시도.
- 결과적으로, 두 인스턴스가 동시에 $y[0]$ 에 값을 쓰게 됨.
- 이것이 바로 ****데이터 레이스(Data Race)****라고 함.

정의되지 않은 동작 (Undefined Behavior)

- 병렬 프로그래밍에서 ****여러 인스턴스(또는 스레드)가 동시에 동일한 메모리 위치에 쓰기(write)****를 하게 되면, 어떤 값이 실제로 저장될지 **예측할 수 없게 됨**.
- 이러한 현상은 프로그램의 **결과가 실행할 때마다 달라지거나, 잘못된 값이 저장되거나, 충돌이 일어나는 등의 오류**를 유발함.

What does this program do?

```
// main C++ code:  
const int N = 1024;  
float* x = new float[N];  
float* y = new float[N];  
  
// initialize N elements of x  
  
// call ISPC function  
shift_negative(N, x, y);
```

```
// ISPC code:  
export void shift_negative(  
    uniform int N,  
    uniform float* x,  
    uniform float* y)  
{  
    foreach (i = 0 ... N)  
    {  
        if (i >= 1 && x[i] < 0)  
            y[i-1] = x[i];  
        else  
            y[i] = x[i];  
    }  
}
```

The output of this program is undefined!

Possible for multiple iterations of the loop body to write to same memory location

Computing the sum of all elements in an array (incorrectly)

배열의 모든 요소의 합을 계산하는 잘못된 방법

What's the error in this program?

```
export uniform float sum_incorrect_1(  
  uniform int N,  
  uniform float* x)  
{  
  float sum = 0.0f;  
  foreach (i = 0 ... N)  
  {  
    sum += x[i];  
  }  
  
  return sum;  
}
```

문제점: sum 변수가 float 타입으로 선언되어 있음. 이는 각 프로그램 인스턴스마다 다른 변수를 의미.

결과: 여러 복사본의 sum 변수를 반환하려고 하기 때문에 컴파일 타임에 타입 오류가 발생합니다.

What's the error in this program?

```
export uniform float sum_incorrect_2(  
  uniform int N,  
  uniform float* x)  
{  
  uniform float sum = 0.0f;  
  foreach (i = 0 ... N)  
  {  
    sum += x[i];  
  }  
  
  return sum;  
}
```

문제점: sum 변수가 uniform float 타입으로 선언되어 있음. 이는 모든 프로그램 인스턴스가 동일한 변수를 공유한다는 의미.

결과: x[i]는 각 프로그램 인스턴스마다 다른 값을 가지므로, 어떤 값이 sum에 복사될지 알 수 없습니다. 이로 인해 컴파일 타임에 타입 오류가 발생합니다.

Computing the sum of all elements in an array (correctly)

```
export uniform float sum_array(  
    uniform int N,  
    uniform float* x)  
{  
    uniform float sum;  
    float partial = 0.0f;  
    foreach (i = 0 ... N)  
    {  
        partial += x[i];  
    }  
  
    // reduce_add() is part of ISPC's cross  
    // program instance standard library  
    sum = reduce_add(partial);  
  
    return sum;  
}
```

* Self-test: If you understand why this implementation correctly implements the semantics of the ISPC gang abstraction, then you've got a good command of ISPC

1.부분 합 계산: 각 프로그램 인스턴스는 partial이라는 개인적인 부분 합을 계산합니다. 이는 각 인스턴스가 배열의 일부 요소를 더하는 것을 의미합니다.

2.reduce_add() 함수 사용: reduce_add() 함수는 모든 인스턴스의 부분 합을 더해 최종 합을 계산합니다. 이 함수는 모든 인스턴스에서 동일한 결과를 반환합니다.

- 부분 합: 각 인스턴스가 독립적으로 부분 합을 계산하므로, 인스턴스 간의 통신이 필요 없음.

- 최종 합 계산: reduce_add() 함수는 모든 인스턴스의 부분 합을 더해 최종 합을 계산합니다. 이 함수는 모든 인스턴스에서 동일한 결과를 반환하므로, 최종 합이 올바르게 계산됩니다.

```
float sum_summary_AVX(int N, float* x) {  
  
    float tmp[8]; // assume 16-byte alignment  
    __m256 partial = _mm256_broadcast_ss(0.0f);  
  
    for (int i=0; i<N; i+=8)  
        partial = _mm256_add_ps(partial, _mm256_load_ps(&x[i]));  
  
    _mm256_store_ps(tmp, partial);  
  
    float sum = 0.f;  
    for (int i=0; i<8; i++)  
        sum += tmp[i];  
  
    return sum;  
}
```

ISPC's cross program instance operations

ISPC는 여러 프로그램 인스턴스가 동시에 실행되는 환경에서 데이터를 처리하기 위해 다양한 연산을 제공합니다. 여기에는 다음과 같은 주요 연산이 포함됩니다:

1.reduce_add():

- 설명: 모든 프로그램 인스턴스의 값을 더하여 하나의 값을 반환합니다.
- 사용 예시: `uniform int64 reduce_add(int32 x);`
- 용도: 배열의 모든 요소의 합을 계산할 때 사용됩니다.

2.reduce_min():

- 설명: 모든 프로그램 인스턴스의 값 중 최소값을 반환합니다.
- 사용 예시: `uniform int32 reduce_min(int32 a);`
- 용도: 배열에서 최소값을 찾을 때 사용됩니다.

3.broadcast():

- 설명: 특정 인스턴스의 값을 모든 인스턴스에 전달합니다.
- 사용 예시: `int32 broadcast(int32 value, uniform int index);`
- 용도: 특정 인스턴스의 값을 다른 모든 인스턴스와 공유할 때 사용됩니다.

4.rotate():

- 설명: 각 인스턴스의 값을 다음 인스턴스로 전달합니다. 인스턴스의 순환 이동을 의미합니다.
- 사용 예시: `int32 rotate(int32 value, uniform int offset);`
- 용도: 인스턴스 간의 값을 순환시키며 전달할 때 사용됩니다.

Compute sum of a variable's value in all program instances in a gang:

```
uniform int64 reduce_add(int32 x);
```

Compute the min of all values in a gang:

```
uniform int32 reduce_min(int32 a);
```

Broadcast a value from one instance to all instances in a gang:

```
int32 broadcast(int32 value, uniform int index);
```

For all i, pass value from instance i to the instance i+offset % programCount:

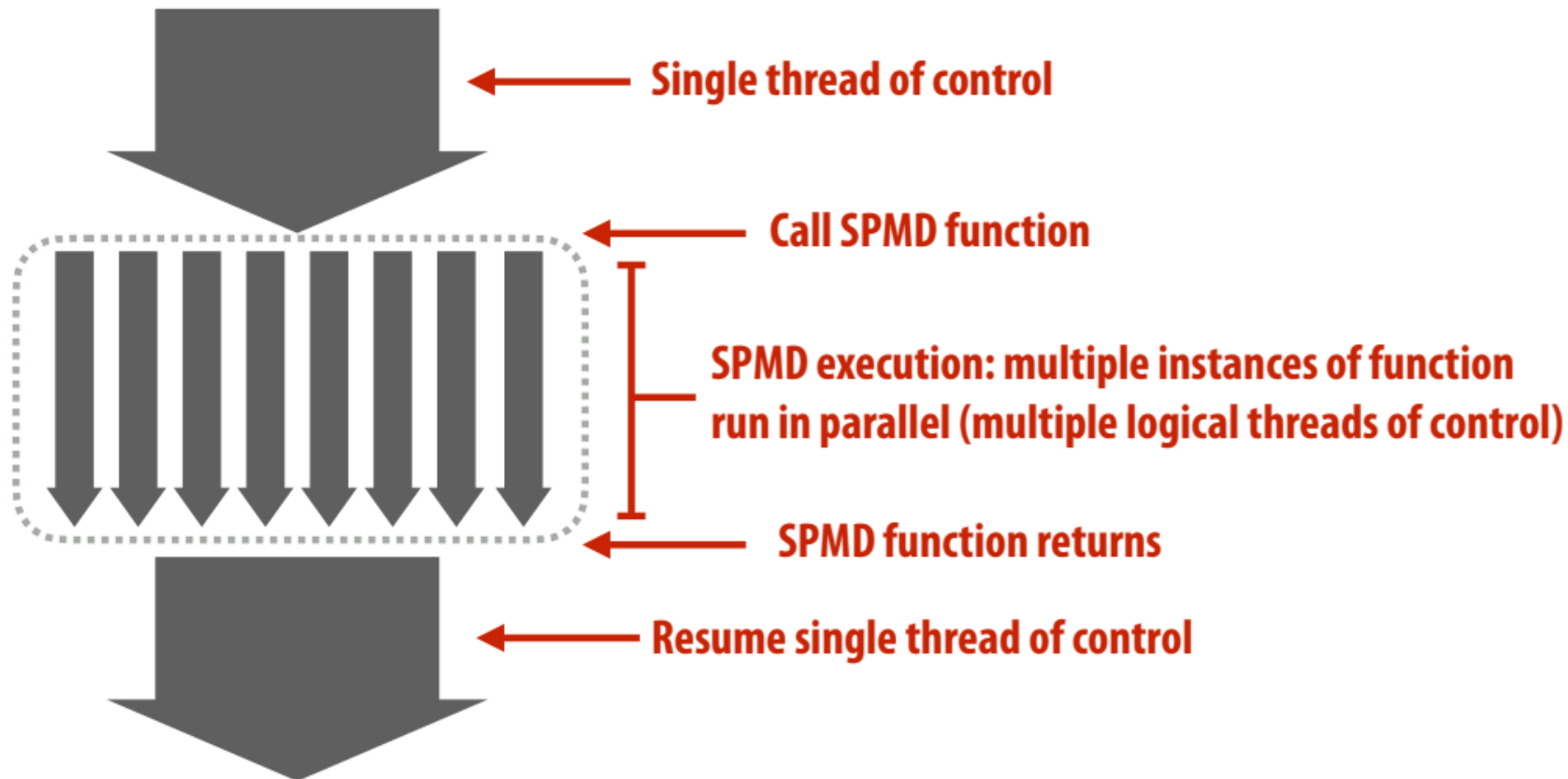
```
int32 rotate(int32 value, uniform int offset);
```

ISPC: abstraction vs. implementation

- 단일 프로그램, 다중 데이터(SPMD) 프로그래밍 모델
 - 프로그래머 “생각”: 갠을 실행하면 ProgramCount 논리적 명령 스트림(각각 다른 값을 갖는 프로그램 인덱스)이 생성됩니다.
 - 이것이 프로그래밍 추상화입니다.
 - 프로그램은 이 추상화의 관점에서 작성됩니다.
- 단일 명령어, 다중 데이터(SIMD) 구현
 - ISPC 컴파일러는 ISPC 갠이 수행하는 로직을 수행하는 벡터 명령어(예: AVX2, ARM NEON)를 방출합니다.
 - ISPC 컴파일러는 조건부 제어 #ow를 벡터 명령어에 매핑(벡터 레인을 마스킹하는 등)하여 처리합니다. 과제 1에서 수동으로 하는 것처럼).
- ISPC의 의미론은 까다로울 수 있습니다.
 - SPMD 추상화 + 균일한 값
(추상화를 통해 구현 세부 사항을 약간 엿볼 수 있음)

SPMD programming model summary

- SPMD = “single program, multiple data”
- Define one function, run multiple instances of that function in parallel on different input arguments



- ISPC에서는 "gang"이라는 추상화를 SIMD 명령어를 통해 구현하며, 이 명령어는 CPU의 한 코어에서 실행되는 하나의 스레드 내에서 작동합니다.
- 이전에 실행된 모든 코드는 사실상 컴퓨터의 네 개 코어 중 단 하나의 코어에서만 실행된 것입니다.
- ISPC는 멀티 코어 실행을 가능하게 하는 "task"라는 또 다른 추상화를 제공합니다. "task"에 대한 자세한 내용은 과제 1을 통해 학습할 수 있습니다.

ISPC는 프로그램 인스턴스 간의 병렬 처리를 위해 태스크라는 개념을 도입합니다. 태스크는 여러 코어에서 동시에 실행될 수 있는 작업 단위입니다.

주요 내용은 다음과 같습니다:

1.태스크 생성:

- 설명: 태스크는 여러 프로그램 인스턴스를 포함하는 작업 단위입니다.
- 사용 예시: launch 키워드를 사용하여 태스크를 생성합니다.
- 용도: 멀티코어 환경에서 병렬 처리를 효율적으로 수행하기 위해 사용됩니다.

2.태스크 실행:

- 설명: 태스크는 여러 코어에서 동시에 실행됩니다.
- 사용 예시: launch 키워드를 사용하여 태스크를 실행합니다.
- 용도: 멀티코어 환경에서 병렬 처리를 효율적으로 수행하기 위해 사용됩니다.

3.태스크 동기화:

- 설명: 태스크가 완료될 때까지 기다리는 동기화 메커니즘이 필요합니다.
- 사용 예시: sync 키워드를 사용하여 태스크를 동기화합니다.
- 용도: 모든 태스크가 완료된 후에 다음 작업을 수행하기 위해 사용됩니다.

Thinking about operating on data in parallel?

- **foreach 사용**: ISPC의 foreach 구문은 병렬 처리를 단순화하여, 마치 순차적으로 실행되는 프로그램처럼 코드를 작성할 수 있게 함.
→ 예를 들어, 배열의 각 요소에 대해 독립적으로 작업을 수행하는 경우를 쉽게 표현할 수 있음

- 예외:

- **균일 변수(Uniform variables)**: 모든 병렬 인스턴스에서 동일한 값을 가지는 변수.
- **Cross-instance 연산**: 병렬 인스턴스 간의 데이터를 공유하거나 합산하는 연산.

- 그러나 ISPC는 저수준 프로그래밍 언어: 프로그래머는 "programIndex" 및 "programCount"를 노출함으로써 각 프로그램 인스턴스가 수행하는 작업과 각 인스턴스가 액세스하는 데이터를 정의할 수 있음.

- 정의되지 않은 출력으로 프로그램을 구현할 수 있음.
- 특정 programCount에 대해서만 올바른 프로그램을 구현할 수 있음

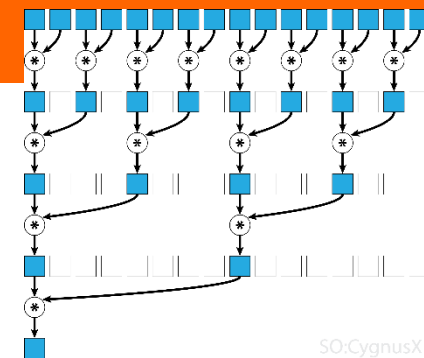
• 동일한 메모리 위치에 여러 스레드가 동시에 쓰기 작업을 수행하면, 어떤 값이 최종적으로 저장될지 알 수 없음.
• 프로그램이 특정한 실행 환경(예: programCount)에 의존하는 경우, 다른 환경에서는 올바르게 작동하지 않을 수 있음.
• 이러한 문제를 방지하려면, 병렬 프로그래밍에서 데이터 의존성과 동기화를 신중히 관리가 필요. 예를 들어, 메모리 충돌을 방지하기 위해 각 스레드가 독립적인 메모리 공간을 사용하거나, 동기화 메커니즘을 도입해야 함.

```
export void ispc_function(  
    uniform int    N,  
    uniform float* x,  
    uniform float* y)  
{  
    foreach (i = 0 ... N)  
    {  
        float val = x[i];  
        float result;  
  
        // do work here to compute  
        // result from val  
  
        y[i] = result;  
    }  
}
```

But can express very advanced cooperation

Here's a program that computes the product of all elements of an array in $\log_2(8) = 3$ steps

배열의 8개 요소 모두의 곱을 $\log_2(8) = 3$ 단계



```
// compute the product of all eight elements in the
// input array. Assumes the gang size is 8.
```

```
export void vec8product(
```

```
    uniform float* x,
```

```
    uniform float* result)
```

```
{
```

```
    float val1 = x[programIndex];
```

```
    float val2 = shift(val1, 1);
```

```
    if (programIndex % 2 == 0)
```

```
        val1 = val1 * val2;
```

```
    val2 = shift(val1, 2);
```

```
    if (programIndex % 4 == 0)
```

```
        val1 = val1 * val2;
```

```
}
```

```
    val2 = shift(val1, 4);
```

```
    if (programIndex % 8 == 0) {
```

```
        *result = val1 * val2
```

```
    }
```

```
}
```

초기화:

float val1 = x[programIndex]; 각 프로그램 인스턴스는 입력 배열 x의 해당 요소로 val1을 초기화합니다. 예를 들어 인스턴스 0은 x[0], 인스턴스 1은 x[1]을 가져오는 식으로.

float val2 = shift(val1, 1); 시프트 함수가 중요합니다. 이것은 교차 인스턴스 연산입니다. 이 함수는 val1의 값을 가져와 지정된 오프셋만큼 이동합니다. 여기서는 1만큼 이동합니다. 따라서 인스턴스 0은 x[1], 인스턴스 1은 x[2], 인스턴스 2는 x[3]을 가져옵니다. 인스턴스 7은 x[0]을 얻습니다(순환 이동으로 생각하면 됩니다).

첫 번째 감소:

if (programIndex % 2 == 0) val1 = val1 * val2; ID가 짝수인 인스턴스(0, 2, 4, 6)는 val1에 수신한 val2를 곱합니다. 홀수 인스턴스는 이 단계에서 아무 작업도 수행하지 않습니다. 실제로는 x[0] * x[1], x[2] * x[3], x[4] * x[5], x[6] * x[7]과 같은 요소 쌍을 곱합니다. val2 = shift(val1, 2); 이번에는 2로 또다시 이동합니다. 따라서 인스턴스 0은 인스턴스 2의 결과를, 인스턴스 2는 인스턴스 4의 결과를 가져오는 식으로 이동합니다.

두 번째 감소:

if (programIndex % 4 == 0) val1 = val1 * val2; 이제 인스턴스 0, 4는 val1에 시프트된 val2를 곱하여 이전 단계의 결과인 (x[0] * x[1]) * (x[2] * x[3]) 및 (x[4] * x[5]) * (x[6] * x[7])을 효과적으로 곱합니다. val2 = shift(val1, 4); 4만큼 시프트.

최종 감소:

if (programIndex % 8 == 0) { *result = val1 * val2 } 인스턴스 0만 최종 곱셈을 수행합니다: (x[0] * x[1] * x[2] * x[3]) * (x[4] * x[5] * x[6] * x[7]), 8개 요소 모두의 곱을 제공합니다. 그런 다음 이 최종 결과를 결과 포인터에 씁니다.

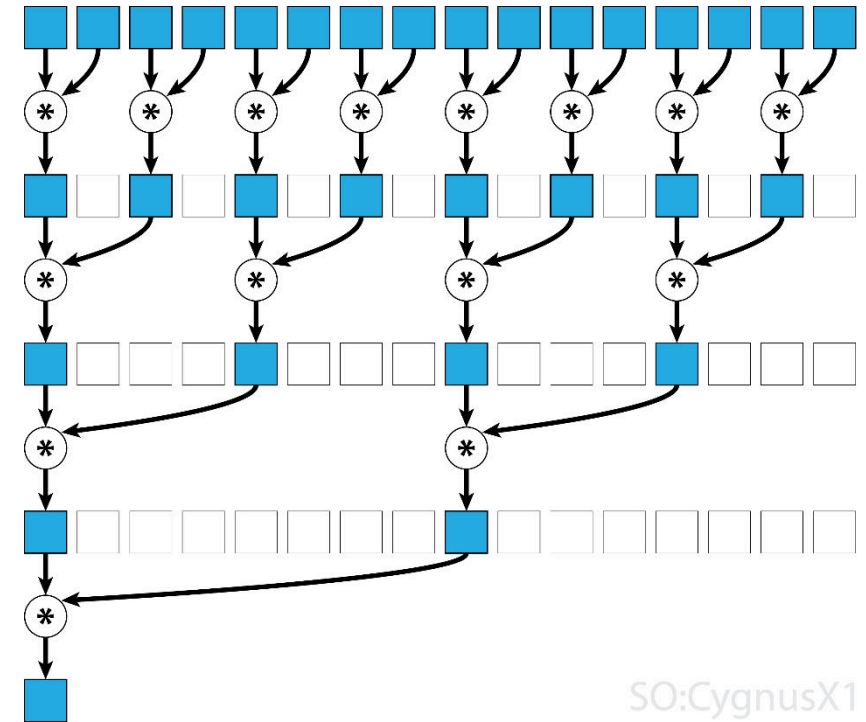
ISPC가 병렬 프로그래밍을 단순화하지만, 필요할 때 이러한 병렬 인스턴스 간의 복잡한 협력을 표현할 수 있는 유연성도 제공함. 이를 위해 프로그래머는 각 인스턴스가 수행하는 작업과 액세스하는 데이터를 정확하게 정의할 수 있는 programIndex 및 programCount에 대한 액세스 권한을 부여할 수 있습니다.

But can express very advanced cooperation

협력: 이 예는 ISPC가 프로그램 인덱스와 인스턴스 간 통신(시프트)을 사용하여 프로그램 인스턴스 간에 세분화된 협업을 가능하게 하는 방법을 제시

효율성: 대수 시간으로 곱을 계산하는 영리한 병렬 알고리즘으로, 대규모 배열의 경우 순차적 접근 방식보다 훨씬 빠릅니다.

낮은 수준의 제어: ISPC는 프로그래머가 더 복잡한 코드를 사용하더라도 성능을 극대화하기 위해 데이터 액세스 및 통신 패턴을 조율할 수 있는 기능을 제공합니다.



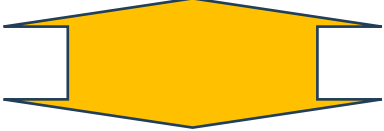
SO: CygnusX1

But what if ISPC was not trying to be a low-level language?

- 프로그래머는 프로그램 인덱스와 프로그램 카운트를 사용하여 병렬 실행을 세밀하게 제어를 통해 최적화가 가능하지만
- 프로그래머는 작업을 병렬 인스턴스 간에 어떻게 분할할지 보다 명확하게 생각해야 함.
- 이러한 낮은 수준의 세부 사항을 숨기고 더 높은 수준의 추상화를 목표로 한다면 ISPC가 어떤 모습 일까..

- 
- 핵심 아이디어는 프로그래머가 프로그램 인덱스나 프로그램 카운트를 직접 처리할 필요가 없는 ISPC를 상상하는 것입니다.
 - 대신, 언어가 병렬성을 표현하기 위해 foreach와 같은 구문을 사용하도록 권장(또는 강제)합니다.
 - 그러면 컴파일러는 이러한 병렬 루프가 실행되는 방식을 책임집니다.

핵심 사항 및 시사점

- 
- 추상화: 추상화 수준을 높여 프로그래머가 하드웨어에서 실행되는 구체적인 방법에 대해 걱정하지 않고 병렬 코드를 더 쉽게 작성할 수 있도록 하는 것이 목표.
 - 프로그래밍 간소화: 프로그램 인덱스와 프로그램 카운트를 제거하면 코드가 순차적 코드에 훨씬 가까워져 오류 발생 가능성이 줄어들고 읽기 쉬워짐.
 - 컴파일러 책임: ISPC 컴파일러는 병렬 실행을 최적화하고 스케줄링하는 데 더 많은 책임을 맡게 됨. **아직까지 성능을 최적화하는데 있어서는 문제가 있음.**

```
export void ispc_function(  
    int    N,  
    float* x,  
    float* y)  
{  
  
    int twoN = 2 * N;  
  
    foreach (i = 0 ... twoN)  
    {  
        float val = x[i];  
        float result;  
  
        // do work here to compute  
        // result from val  
  
        y[i] = result;  
    }  
}
```

Another alternative

- “배열 인덱싱을 완전히 없애면 어떨까요?”
- 메모리에서 데이터에 액세스하는 방법에 대한 세부 사항을 추상화하는 것입니다.
- 프로그래머가 루프를 작성하고 배열에 수동으로 인덱싱하는 대신 언어 자체가 ‘컬렉션’ 내에서 데이터의 병렬 처리를 처리하는 것입니다.
- 컬렉션: ‘컬렉션’은 배열, 목록 또는 벡터와 유사하게 여러 데이터 요소를 보유하는 일반적인 데이터 구조라고 생각하면 됩니다.
- 핵심은 프로그래머가 이 컬렉션이 어떻게 저장되거나 액세스되는지 자세히 알 필요가 없다는 것입니다.

요소별 연산: 프로그래머는 단일 요소에 대해 작동하는 함수를 정의합니다. 그러면 언어가 이 함수를 컬렉션의 모든 요소에 자동으로 병렬로 적용함.

추상화: 이 접근 방식은 매우 높은 수준의 추상화를 제공합니다. 프로그래머는 루프를 병렬화하거나 개별 요소에 액세스하는 방법이 아니라 각 요소에서 어떤 연산을 수행할지에 집중함.

NumPy/PyTorch와 유사합니다: 이 모델이 NumPy(Python의 수치 연산용) 및 PyTorch(머신 러닝용) 같은 라이브러리의 작동 방식과 매우 유사함

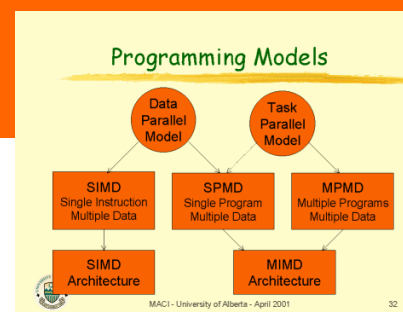
```
float dowork(float x) {  
    // do work here to compute  
    // result from x  
}  
  
Collection x; // data structure of N  
  
// invoke dowork for all elements of x,  
// placing results in collection y  
Collection y = map(dowork, x, y);
```

```
import numpy as np  
  
def addOne(i):  
    return i+1  
mapAddOne = np.vectorize(addOne);  
  
X = np.arange(15) # create numPy array [0, 1, 2, 3, ...]  
Y = np.arange(15) # create numPy array [0, 1, 2, 3, ...]  
  
Z = X + Y;          # Z = [0, 2, 4, 6, ... ]  
Zplus1 = mapAddOne(Z); # Zplus1 = [1, 3, 5, 7, ...]
```

● 프로그래밍 모델: 생각하는 방법

➔ "프로그래밍 모델은 병렬 프로그램 구성에 대해 생각할 수 있는 방법을 제공."

- 기본적으로 프로그래밍 모델은 병렬 코드를 구성하는 방법에 대한 프레임워크를 제공합니다. 건물의 건축 청사진이라고 생각하자.
- 청사진은 모든 벽돌을 정확히 어떻게 놓아야 하는지 알려주지는 않지만 전체적인 설계(예: 방의 위치, 배관의 위치)를 알려줍니다.
- 병렬 프로그래밍에서는 ISPC에서 사용하는 SPMD(단일 프로그램, 다중 데이터)와 같은 모델을 사용하면 문제를 동시에 실행할 수 있는 작은 작업으로 나누는 방법을 생각할 수 있습니다. 다른 모델도 존재하며, 각 모델마다 병렬성을 구성하는 고유한 방식이 있습니다.



● 추상화 대 구현

➔ "여러 가지 유효한 구현을 허용하는 추상화를 제공."

- 추상화는 복잡한 세부 사항을 숨기고 무언가를 표현하는 단순화된 방식입니다.
- 자동차의 핸들을 생각보자. 핸들은 자동차의 바퀴를 돌리는 복잡한 메커니즘을 추상화한 것입니다.
- 기어와 링크지가 어떻게 작동하는지 정확히 알 필요 없이 스티어링 휠을 사용할 수 있습니다.
- 병렬 프로그래밍에서 추상화는 프로그래밍 모델에서 제공하는 기능입니다(앞서 설명한 foreach나 map함수 등). 이러한 추상화를 사용하면 하드웨어가 어떻게 실행해야 하는지 정확히 지정하지 않고도 병렬성을 표현할 수 있습니다.
- Map함수는 기본 하드웨어(SIMD 명령어, 멀티코어 처리 등)에 따라 다른 병렬 실행 전략을 사용하여 구현할 수 있습니다.